

UNIVERSITÉ JOSEPH KI-ZERBO
(UJKZ)

INSTITUT BURKINABÈ DES ARTS ET MÉTIERS
(IBAM)



RAPPORT DE STAGE
POUR L'OBTENTION DE LA LICENCE PROFESSIONNELLE

OPTION : Méthodes Informatiques Appliquées à la Gestion (MIAGE)

Période de stage : 01 Juillet au 30 Septembre 2025

THÈME :

**CONCEPTION, DÉVELOPPEMENT ET INTÉGRATION D'UN MODULE DE
GESTION DES EXPRESSIONS DE BESOINS DANS L'ERP DOLIBARR : CAS
DE L'URCN**

Présenté par :

Armand Hendrix OUEDRAOGO

Maître de stage :

Dr. VALEA Innocent

Responsable Qualité à l'URCN

Directeur de rapport :

Dr. OUATTARA Yacouba

Maître Assistant en Informatique à
l'IBAM

Année académique 2024–2025

SOMMAIRE

SOMMAIRE	i
DEDICACE	ii
REMERCIEMENTS	iii
LISTE DES SIGLES ET ABREVIATIONS	iv
LISTE DES FIGURES	vi
INTRODUCTION GENERALE	1
1 PRESENTATION DES STRUCTURES DE FORMATION ET D’ACCUEIL . .	2
2 ANALYSE ET CONCEPTION	9
3 BILAN DU STAGE	66
CONCLUSION GENERALE	69
BIBLIOGRAPHIE	70
WEBOGRAPHIE	71
TABLE DES MATIERES	72
ANNEXES	I

DEDICACE

*Nous dédions ce modeste travail à notre famille,
source de courage, de patience et d'inspiration.*

REMERCIEMENTS

Il nous est particulièrement agréable d'exprimer, au terme de ce stage, notre profonde gratitude à toutes les personnes et institutions qui, de près ou de loin, ont contribué à la réalisation de ce travail.

Nous tenons tout d'abord à adresser nos sincères remerciements au Pr. BAYILI Gilbert, Directeur de l'IBAM, pour son engagement constant en faveur de la qualité de la formation.

Nos remerciements s'adressent également à notre maître de stage, Dr. VALEA Innocent, Responsable Qualité à l'URCN, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de cette expérience.

Nous exprimons aussi toute notre reconnaissance à notre directeur de mémoire, Dr. OUAT-TARA Yacouba, Maître Assistant en Informatique, pour ses orientations pertinentes et son soutien académique.

Nous n'oublions pas le personnel de l'URCN, qui nous a réservé un accueil chaleureux et qui a facilité notre intégration dans un cadre professionnel stimulant.

Enfin, nous adressons notre gratitude à toutes celles et ceux qui, de près ou de loin, ont contribué à la réussite de ce stage et à l'élaboration de ce rapport.

LISTE DES SIGLES ET ABRÉVIATIONS

Sigles / Abréviations	Significations
2TUP	Two (2) Tracks Unified Process
ABF	Assurance Banque Finance
ATOS	Administratifs, Techniques, Ouvriers, de Service
CNRST	Centre National de la Recherche Scientifique et Technologique
COCOMO	Constructive Cost Model
CSAF	Chef du Service Administratif et Financier
CSS	Cascading Style Sheets
CU	Cas d'Utilisation
DA	Directeur Adjoint
EPESCT	Etablissement Public de l'Etat à Caractère Scientifique, Culturel et Technique
ERP	Enterprise Resource Planning
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IBAM	Institut Burkinabè des Arts et Métiers
IHM	Interface Homme-Machine
INERA	Institut de l'Environnement et de la Recherche Agricole
INSS	Institut des Sciences des Sociétés
IRSAT	Institut de Recherche en Sciences Appliquées et Technologies
IRSS	Institut de Recherche en Sciences de la Santé
LMD	Licence Master Doctorat
MBF	Master en Banque Finance
MIAGE	Méthodes Informatiques Appliquées à la Gestion

Suite à la page suivante...

Sigles / Abréviations	Significations
MVC	Model Vue Controller
PHP	PHP Hypertext Preprocessor
RUP	Rational Unified Process
SQL	Structured Query Language
UJKZ	Université Joseph KI-ZERBO
UML	Unified Modeling Language
URCN	Unité de Recherche Clinique Nanoro
XP	Extreme Programming

LISTE DES FIGURES ET GRAPHIQUES

1.1	Organigramme de l'IBAM	3
1.2	Organigramme de l'URCN	7
2.1	Schéma présentant le processus 2TUP	15
2.2	Planning prévisionnel de réalisation du projet	17
2.3	Diagramme des cas d'utilisation	26
2.4	Diagramme de séquence du cas d'utilisation « S'authentifier »	33
2.5	Diagramme de séquence du cas d'utilisation « Créer une expression »	34
2.6	Diagramme de séquence du cas d'utilisation « Statuer expression premier niveau »	35
2.7	Diagramme de séquence du cas d'utilisation « Statuer expression deuxième niveau »	36
2.8	Diagramme de séquence du cas d'utilisation « Traiter une expression »	37
2.9	Diagramme de séquence du cas d'utilisation « Notifier utilisateur »	38
2.10	Architecture 3 tiers ou à 3 niveaux	40
2.11	Diagramme des classes	46
2.12	Diagramme de déploiement	48
2.13	Structure de l'architecture Modèle–Vue–Contrôleur	51
2.14	Écran de connexion	52
2.15	Écran de création d'expression	53
2.16	Écran de validation d'expression	54
2.17	Écran de traitement d'expression	54
2.18	Résultat obtenu	57
2.19	Planning réel	66
3.1	Formulaire d'expression de besoin utilisé à l'URCN	VIII

LISTE DES TABLEAUX

2.1	Comparaison des principales méthodologies de développement	12
2.2	Exigences fonctionnelles du module de gestion des expressions de besoin	20
2.3	Liste des cas d'utilisation du système	24
2.4	Description du CU01 « S'authentifier »	27
2.5	Description du CU02 « Créer une expression »	28
2.6	Description du CU03 « Statuer expression premier niveau »	29
2.7	Description du CU04 « Statuer expression deuxième niveau »	30
2.8	Description du CU05 « Traiter une expression »	31
2.9	Description de la classe « User »	41
2.10	Description de la classe « Notification »	42
2.11	Description de la classe « ExpressionBesoin »	42
2.12	Description de la classe « ExpressionBesoinFingerprint »	43
2.13	Description de la classe « ItemExpressionBesoin »	44
2.14	Description de la classe « FournisseurRecommande »	44
2.15	Jeu de test unitaire prévisionnel	56
2.16	Jeu de test unitaire final	57
2.17	Jeu de test fonctionnel	59
2.18	Matrice de couverture des tests par exigence fonctionnelle	60
2.19	Comparaison des méthodes d'estimation de coût logiciel	64
2.20	Coût total du projet	65
3.1	Formules de calcul selon la méthode COCOMO	II
3.2	Calcul basé sur la réalisation effective	III
3.3	Comparaison des approches d'estimation	V

INTRODUCTION GENERALE

Dans un contexte marqué par des besoins croissants en solutions numériques adaptées, les institutions de recherche en santé cherchent à renforcer leurs capacités organisationnelles et administratives. La gestion efficace des ressources et des demandes internes constitue un atout pour améliorer la qualité du travail mené. Les systèmes intégrés de gestion, tels que les ERP, apparaissent alors comme des outils incontournables pour répondre à ces exigences.

L'Unité de Recherche Clinique de Nanoro (URCN), qui est de nos jours la Direction Régionale du Centre-Ouest de l'Institut de Recherche en Sciences de la Santé (IRSS), s'inscrit dans cette dynamique. Elle mène des travaux scientifiques dédiés à la recherche en santé et contribue activement à la lutte contre diverses maladies. Cependant, comme toute organisation, l'URCN doit relever plusieurs défis liés à la gestion de ses besoins internes. La coordination et le suivi des demandes exprimées par ses différents services et par ses membres constituent un enjeu majeur pour garantir l'efficacité de ses opérations.

C'est dans ce cadre que nous avons effectué notre stage de fin de cycle au sein de l'URCN. La mission qui nous a été confiée portait sur la conception, le développement et l'intégration d'un module de gestion des expressions de besoins dans l'ERP Dolibarr, cas de l'URCN, avec Dolibarr déjà en usage au sein de la structure. Ce projet vise à offrir un outil pratique, capable d'optimiser le processus de validation, de traitement et de suivi des demandes formulées par les différents acteurs de l'unité.

Le présent rapport rend compte de l'expérience professionnelle que nous avons vécue au sein de l'URCN. Il présente la démarche adoptée, les outils mobilisés et les résultats obtenus au cours de la mission. Pour une meilleure compréhension, le rapport est structuré en trois grands chapitres. Le premier chapitre est consacré à la présentation de notre structure de formation et de la structure d'accueil. Le deuxième chapitre aborde l'analyse et la conception du système. Le troisième chapitre présente le bilan du stage effectué à l'URCN.

Chapitre 1

PRESENTATION DES STRUCTURES DE FORMATION ET D'ACCUEIL

Ce chapitre a pour objectif de présenter nos structures de formation et d'accueil. Dans un premier temps, nous présenterons l'Institut Burkinabè des Arts et Métiers (IBAM), puis dans un second temps, nous décrirons l'Unité de Recherche Clinique de Nanoro (URCN).

I. Présentation de la structure de formation

Dans cette partie, nous dresserons une présentation générale de l'IBAM en faisant ressortir son historique, ses objectifs, son organisation et ses offres de formations.

1. Historique

L'IBAM est un établissement d'enseignement professionnel. Il a été créé en janvier 2000 à la faveur de la refondation de l'Université de Ouagadougou rebaptisée l'Université Ouaga I Pr. Joseph KI-ZERBO depuis le 26 Décembre 2015, puis Université Joseph KI-ZERBO depuis le 12 Avril 2019. Elle est la matérialisation, la concrétisation dans les faits de l'engagement de l'UJKZ dans la professionnalisation des filières. Cette nouvelle vision a pour but d'accroître son efficacité externe.

2. Objectifs

Actuellement sous la direction du Pr Gilbert BAYILI, l'objectif principal de l'IBAM est de répondre aux besoins du marché de l'emploi en mettant à sa disposition un fort potentiel humain formé de cadres moyens et supérieurs dans divers secteurs d'activités.

3. Organisation

L'IBAM est organisé en deux organes à savoir l'organe statutaire et l'organe d'exécution. L'organisation des enseignants et l'organisation du contrôle des aptitudes et des connaissances sont fixées par arrêté ministériel. L'institut est dirigé par un directeur assisté d'un adjoint, tous deux élus par le collège électoral de l'établissement pour un mandat de deux (2) ans.

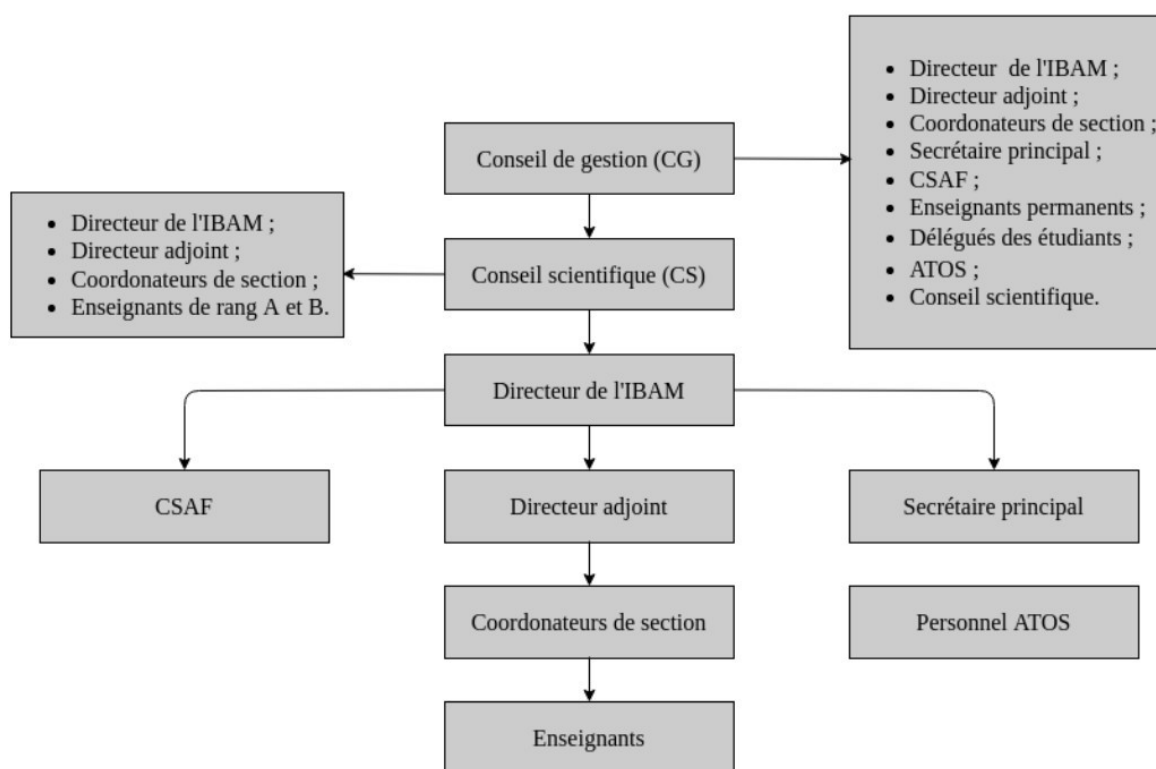


FIGURE 1.1 – Organigramme de l'IBAM

a. Organe statutaire

L'organe statutaire de l'IBAM est composé du Conseil de gestion et du Conseil scientifique. Le Conseil de gestion est l'organe de l'institut. Il est composé du Directeur de l'IBAM, du DA, des Coordonnateurs de sections, du Secrétaire principal, du CSAF, des Enseignants permanents, de deux (02) représentants des Délégués de syndicats des étudiants et d'un représentant du personnel ATOS. Il est présidé par le Directeur de l'IBAM et se réunit tous les trois (03) mois sauf en cas de session extraordinaire sur convocation de son président. Le Conseil de gestion a pour missions de :

- définir le règlement intérieur de l'institut
- proposer le budget de fonctionnement de l'institut

- décider des mesures administratives et financières conformément aux textes en vigueur
- proposer la modification des statuts de l'IBAM

Le Conseil scientifique est composé également du Directeur de l'IBAM, du DA, des Coordonnateurs de section, des enseignants de rang A et les enseignants de rang B de l'Institut. Il est présidé par le Directeur de l'IBAM. Il se réunit et délibère dans les mêmes conditions que le Conseil de gestion mais son secrétariat est assuré par un des Coordonnateurs de section. Le Conseil scientifique a pour mission de :

- proposer une organisation générale des enseignements de l'institut
- proposer l'ouverture, la fusion ou la fermeture de filières
- créer des titres, des diplômes et d'examiner les équivalences pédagogiques
- proposer la création de nouveaux postes et d'apprécier les orientations de la recherche au sein de l'institut

b. Organe d'exécution

L'organe d'exécution a pour mission de mettre en pratique les décisions prises par l'organe statutaire. Il est composé essentiellement de la Direction de l'IBAM. La Direction de l'IBAM est pilotée par un Directeur dont les missions sont de diriger, de contrôler le fonctionnement des services et de définir la politique de l'institut. Elle coordonne les activités des différents services et veille au maintien de l'image de marque de l'Institut. En outre, elle veille à la mise en œuvre des décisions, des projets et des programmes. Enfin, elle veille à la mise en application des textes et règlements de l'établissement. Le Directeur de l'IBAM est assisté par une Secrétaire de direction. Rappelons que le Secrétaire principal et le CSAF pour leur part, assistent également le Directeur de l'IBAM dans l'accomplissement de certaines activités en relation avec leurs domaines de spécialisation. La Direction est composée du Secrétariat de direction, du Service administratif et financier, et du Secrétariat principal.

4. Filières de formation

En vue d'atteindre les objectifs cités précédemment, l'IBAM offre des formations dans plusieurs filières réparties en deux (2) groupes selon les diplômes, à savoir le groupe des Licences professionnelles et celui des Masters.

a. Licences professionnelles

Les filières de formations initiales en licences professionnelles sont :

- Assurance Banque Finance (ABF)
- Assistant de Direction/Bilingue (ADB)
- Assistant de Direction Comptable (ADC)
- Comptabilité-Contrôle-Audit (CCA)
- Marketing Innovation DigitalE (MID)
- Informatique, option Méthodes Informatiques Appliquées à la Gestion (MIAGE)
- Informatique, option Réseaux et Télécommunications (RI-MT)

b. Masters

En master, l'IBAM offre cinq (05) filières de formation qui sont :

- Master en Administration et Gestion des Entreprises (MAGE)
- Master en Banque-Finance (MBF)
- Master en Comptabilité-Contrôle-Audit (MCCA)
- Master en Informatique, option Ingénierie des Systèmes d'Informations en Entreprise (M2ISIE)
- Master en Informatique, option Sécurité Informatique (MISI)

La filière MIAGE a été créée dans l'optique de répondre aux besoins croissants des entreprises en cadres compétents dans le domaine de la technologie et des techniques informatiques. Cette filière accueille en première année les bacheliers des séries C, D et E ayant passé avec succès le test de sélection ou ayant été acceptés en tant qu'auditeurs libres. Les étudiants en fin de cycle en MIAGE doivent mettre en pratique les connaissances acquises en classe en effectuant un stage d'une durée d'au moins trois (03) mois suivi d'une soutenance publique. Ce stage a pour objectif de leur permettre de se familiariser avec le monde professionnel. C'est dans ce cadre que nous avons été accueilli au sein de l'URCN que nous présenterons dans les lignes qui suivent.

II. Présentation de la structure d'accueil(URCN)

L'URCN, structure d'accueil de notre stage, fera l'objet d'une présentation détaillée dans les pages qui suivent, notamment en ce qui concerne son positionnement national, son historique, ses missions et son organisation interne.

1. Positionnement sur le plan national

L'URCN est un établissement public de recherche placé sous la tutelle de la Direction Régionale du Centre-Ouest de l'IRSS. L'IRSS est l'un des quatre instituts spécialisés rattachés

au CNRST. Le CNRST est un EPESCT. Il est chargé de coordonner et de contrôler l'ensemble des activités de recherche et d'innovation mené par les quatre instituts placés sous sa responsabilité, à savoir : l'INSS, l'INERA, l'IRSS et l'IRSAT.

2. Histoire de l'URCN

L'URCN a été officiellement créée en avril 2009 par le ministre de la Santé de l'époque, Monsieur Seydou Bouda. Cette initiative s'inscrivait dans le cadre de la mise en œuvre de l'essai vaccinal contre le paludisme, réalisé auprès de plus de 1281 enfants de la zone de Nanoro. Depuis lors, l'unité s'est imposée comme un acteur majeur de la recherche en santé dans la région.

3. Missions et objectifs de l'URCN

La mission principale de l'URCN est de conduire des recherches cliniques et biomédicales de haut niveau, tout en assurant la formation des jeunes scientifiques. L'unité contribue ainsi à l'amélioration et à la rationalisation des soins de santé, en particulier dans les pays tropicaux. Une attention particulière est accordée à la lutte contre le paludisme, qui demeure l'une des priorités de santé publique dans la région.

4. Organisation structurelle de l'URCN

L'URCN est organisée en deux grands pôles qui structurent ses activités : le pôle recherche et formation et le pôle appui à la recherche.

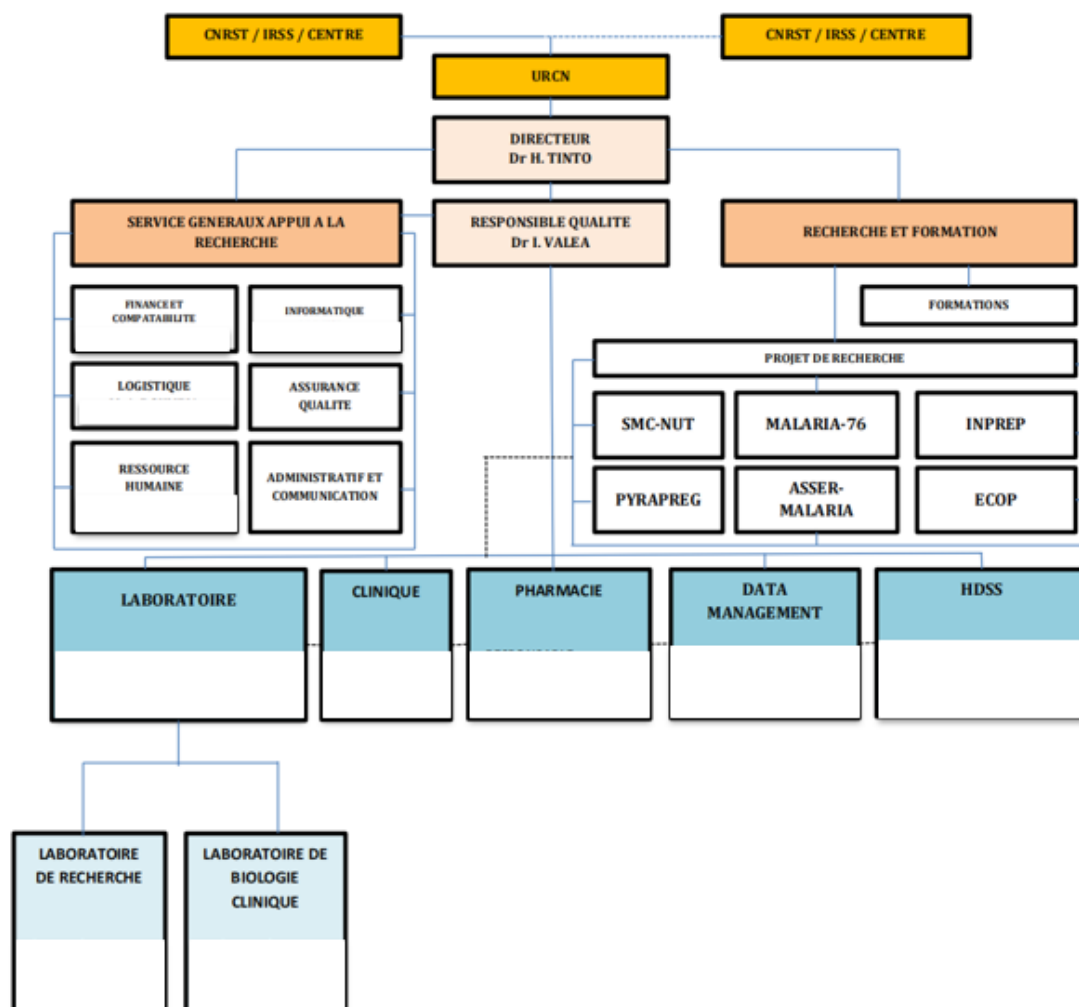


FIGURE 1.2 – Organigramme de l'URCN

a. Pôle recherche et formation

Le pôle recherche et formation est centré sur la conduite des projets scientifiques et sur le développement des compétences. Il se subdivise en deux axes principaux : la coordination des études et celle de la formation. La coordination des études assure le suivi et la supervision des activités de recherche, en veillant à la conformité des travaux avec les protocoles établis et les normes scientifiques en vigueur. Quant à la coordination de la formation, elle a pour rôle d'encadrer et d'accompagner les étudiants dans leurs travaux de recherche ou leurs formations continues. Elle définit également une politique de formation adaptée aux différents profils accueillis par l'unité. Le pôle recherche et formation comprend plusieurs services spécialisés qui soutiennent les activités scientifiques, notamment la clinique, le laboratoire, le service de data management, la pharmacie et le système de surveillance démographique et de santé.

b. Pôle appui à la recherche

Le pôle appui à la recherche regroupe les services qui assurent le bon fonctionnement administratif et technique de l'unité. Il comprend six départements : les finances et la comptabilité, les ressources humaines, la qualité, l'administration et la communication, la logistique et enfin le service informatique. Ce dernier a constitué le cadre d'accueil de notre stage.

c. Le Service Informatique

Le service informatique de l'URCN est placé sous la responsabilité du Dr Toussaint Rouamba, médecin épidémiologiste, biostatisticien et maître de recherche. Ce service occupe une place stratégique dans la structure, car il assure le bon fonctionnement des infrastructures numériques et des systèmes d'information. Ses missions couvrent un large éventail d'activités. Il assure la mise en place, la maintenance et la gestion du matériel ainsi que des logiciels utilisés dans l'unité. Il garantit le fonctionnement optimal des réseaux et des systèmes d'information tout en apportant une assistance technique et une formation aux utilisateurs. Le service informatique est également responsable de l'administration des serveurs, de la sécurisation des réseaux locaux et du développement ou de la maintenance de logiciels et bases de données spécifiques aux besoins de l'unité. Il s'occupe aussi de la gestion du courant faible (onduleurs), de la coordination des accès Internet pour l'ensemble des sites de l'URCN, ainsi que du renouvellement et de l'installation des postes informatiques.

Chapitre 2

ANALYSE ET CONCEPTION

Ce chapitre s'ouvre après la présentation des structures de formation et d'accueil. Il est articulé en deux volets. Le premier volet expose le thème retenu pour notre étude, la démarche d'analyse et de conception adoptée, les acteurs impliqués ainsi que le planning de réalisation du projet. Le second volet se consacre à l'étude technique et à la conception détaillée du module. Enfin, nous présentons les différents éléments mobilisés pour la mise en œuvre du projet.

I. Etude Prealable

Cette section vise à présenter de manière détaillée le thème du projet, la méthode d'analyse et de conception adoptée, les membres du groupe de travail ainsi que le planning de réalisation.

1. Présentation du thème

La présentation de notre thème s'articule autour de trois axes principaux : une étude préliminaire, la problématique identifiée, et les résultats attendus.

a. Préliminaire

Dans le contexte actuel marqué par une digitalisation croissante des organisations, la gestion efficace des ressources et des processus internes est devenue une nécessité incontournable. Les institutions de recherche, à l'instar de l'Unité de Recherche Clinique de Nanoro (URCN), font face à des besoins multiples et variés émanant de leurs différents services. La prise en charge rapide et transparente de ces besoins constitue un facteur clé pour garantir la fluidité des opérations et l'efficacité globale de l'organisation. Un processus peut être défini comme un enchaînement d'activités coordonnées permettant d'atteindre un objectif précis et validé par l'ensemble des acteurs concernés. Lorsqu'il s'agit de gérer des besoins internes, l'absence d'un système centralisé entraîne souvent des lenteurs, des erreurs et un manque de traçabilité ; ce qui affecte la performance des services. L'URCN, qui dispose déjà de l'ERP Dolibarr, reconnaît la nécessité d'optimiser son mode de gestion des expressions de besoins. En effet, un module

intégré au sein de cet ERP permettrait de centraliser les demandes, d'automatiser certaines étapes de validation et de faciliter le suivi par l'ensemble des acteurs concernés. Les objectifs recherchés à travers l'intégration d'un tel module sont :

- **L'efficacité opérationnelle** : réduire les délais et améliorer la fluidité dans le traitement des demandes.
- **La traçabilité et la transparence** : assurer un meilleur suivi et une visibilité claire sur l'état des besoins exprimés.
- **La cohérence et la fiabilité** : minimiser les erreurs humaines et garantir une gestion harmonisée des demandes.
- **L'optimisation des ressources** : permettre une meilleure allocation et une meilleure planification des moyens disponibles.

Ainsi, le présent projet s'inscrit dans une logique d'innovation et de modernisation des pratiques de gestion à l'URCN. Il vise à renforcer la capacité du service informatique à soutenir les missions de recherche et d'appui administratif en proposant un outil numérique adapté, fiable et durable.

b. Problématique

Au sein de l'Unité de Recherche Clinique de Nanoro (URCN), la gestion des besoins exprimés par les différents services constitue un enjeu stratégique pour assurer le bon fonctionnement de ses activités de recherche et d'appui administratif. Actuellement, le traitement de ces expressions de besoins se heurte à plusieurs difficultés : absence d'un système centralisé, lenteurs dans la validation et le suivi, manque de traçabilité, et risques d'erreurs liés aux traitements manuels. Ces insuffisances entraînent une perte de temps, une faible visibilité des demandes et parfois une mauvaise allocation des ressources.

Problématique centrale : *Comment concevoir et intégrer un module de gestion des expressions de besoins dans l'ERP Dolibarr afin d'améliorer, au sein de l'URCN, la coordination, la traçabilité et l'efficacité du traitement des demandes ?*

C'est pour répondre à cette problématique que nous avons retenu le thème suivant :

« Conception, développement et intégration d'un module de gestion des expressions de besoins dans l'ERP Dolibarr : cas de l'URCN »

Ce projet vise à fournir à l'URCN un outil intégré, pratique et fiable, permettant d'optimiser la gestion des demandes internes, d'améliorer la traçabilité des processus et de renforcer l'efficacité de ses opérations.

c. Résultats attendus

Face aux besoins exprimés par l'URCN, l'objectif principal est de concevoir un module intégré à l'ERP Dolibarr permettant de gérer de manière optimale les différentes expressions de besoins formulées par les services de l'unité. Ce module devra répondre à plusieurs exigences fonctionnelles clés, notamment :

- Création, validation et traitement des expressions de besoins
- Gestion des utilisateurs habilités à soumettre, valider et traiter ces expressions
- Association des services et des projets en cours aux expressions de besoins
- Centralisation et traçabilité des expressions pour renforcer la transparence
- Automatisation partielle du processus de validation afin de réduire les délais et limiter les erreurs
- Accès au suivi de l'état d'avancement des expressions
- Génération d'un document officiel représentant l'expression de besoin, garantissant sa fiabilité
- Consultation de statistiques liées aux expressions de besoins

Pour atteindre ces objectifs, une démarche méthodique a été mise en œuvre, reposant sur des techniques d'analyse et de conception éprouvées. Cette approche vise à assurer la fiabilité, la cohérence et la pérennité du module développé.

2. Méthode d'analyse et conception

Une méthode d'analyse et de conception vise à structurer les étapes initiales du développement d'un système, afin d'assurer une meilleure adéquation entre la solution finale et les besoins réels du client. Elle repose sur la formalisation progressive du besoin, à partir d'un énoncé informel tel qu'exprimé par le client enrichi par des recherches complémentaires auprès des experts du domaine fonctionnel, notamment les futurs utilisateurs. Cette démarche inclut également l'étude de l'existant, c'est-à-dire l'analyse des processus actuellement en place chez le client, que le système devra prendre en charge ou améliorer. La phase d'analyse permet de définir précisément les attentes en matière de fonctionnalités, de performance, de fiabilité, de maintenance, de sécurité, d'évolutivité et d'interopérabilité. La phase de conception, quant à elle, consiste à modéliser de manière claire et non ambiguë le fonctionnement futur du système, généralement à l'aide de langages de modélisation, afin de faciliter sa mise en œuvre technique.

a. Le processus de développement

Le processus de développement désigne l'ensemble des activités organisées de manière structurée, visant à atteindre les objectifs d'un projet de création ou d'évolution d'un Système d'Information (SI). Il existe plusieurs approches méthodologiques pour conduire ce type de projet, parmi lesquelles cinq modèles se distinguent par leur popularité et leur usage : CASCADE, RUP, 2TUP, SCRUM et XP.

Chacun de ces processus présente des caractéristiques propres en termes de cycle de vie, de gestion des risques, de flexibilité et d'implication des parties prenantes. Le tableau 2.1 propose une synthèse comparative de ces cinq méthodes, afin de mieux cerner leurs avantages, leurs limites et leurs contextes d'application.

TABLE 2.1 – Comparaison des principales méthodologies de développement

Méthodologie	Description	Atouts	Insuffisances
RUP	Méthodologie et outil prêt à l'emploi. Cible des projets de plus de 10 personnes.	Itératif. Spécifie le dialogue entre les intervenants.	Mise en œuvre floue. Ne couvre pas les phases amont et aval.
Cascade	Phases déroulées de manière séquentielle.	Phases du projet bien distinguées.	Non itératif. Pas de modèles de documents.
2TUP	Centré sur l'architecture. Cycle de développement en Y. Cible tous types de projets.	Itératif, orienté technologie et gestion des risques. Définit profils, plannings, prototypes.	Superficiel sur les phases amont/aval. Pas de documents types.
XP	Bonnes pratiques de développement. Cible : moins de 10 personnes.	Itératif, axé technique. Programmation en duo.	Mise en œuvre floue. Ne couvre pas les phases amont et aval.
Scrum	Itérations appelées sprints.	Confiance aux développeurs. Chaque sprint produit une fonctionnalité testée.	Mise en œuvre non précisée. Développement rapide et répétitif.

L'étude comparative menée sur les processus de développement RUP, Cascade, 2TUP, XP et

Scrum, résumée dans le tableau 2.1, nous a permis de tirer les conclusions suivantes :

- **RUP** : ce processus néglige les contraintes techniques, pourtant indispensables à notre projet. Il a donc été écarté.
- **Cascade** : il s'agit d'un processus séquentiel, non itératif, ce qui limite sa flexibilité.
- **2TUP** : il offre l'avantage de séparer les contraintes fonctionnelles et techniques en deux branches distinctes, pouvant être exploitées en parallèle.
- **XP** : il met l'accent presque exclusivement sur la phase de développement, au détriment de la capture des besoins et de la conception, ce qui le rend inadapté à notre projet.
- **Scrum** : il divise le projet en sprints, dont la durée ne dépasse théoriquement pas 30 jours.

À la lumière de cette analyse comparative, et dans l'objectif de maîtriser les risques tout en assurant la réussite de notre projet, nous avons choisi d'adopter le processus **2TUP**.

Ce choix se justifie par plusieurs raisons :

- D'une part, le 2TUP accorde une place essentielle à la technologie, aspect crucial pour notre projet.
- D'autre part, en tant que processus en « Y », il distingue trois branches : la branche fonctionnelle, la branche technique et la branche réalisation. Les branches fonctionnelle et technique peuvent être menées en parallèle. Ainsi :
 - Si la technologie évolue ou qu'un besoin technique est modifié, il est possible de traiter la branche technique indépendamment puis de la réintégrer facilement au projet.
 - Si une nouvelle fonctionnalité apparaît, seule la branche fonctionnelle est affectée, sans incidence sur la branche technique.

Le 2TUP, qui implémente le processus unifié, propose un cycle de développement en « Y » (voir Figure 2.1), dissociant les aspects techniques des aspects fonctionnels. Il débute par une étude préliminaire visant à :

- Identifier les acteurs interagissant avec le système
- Recenser les échanges entre acteurs et système
- Élaborer le cahier des charges
- Modéliser le contexte

1. Pour en savoir plus sur les méthodologies de développement logiciel, voir :
– <https://www.ibm.com/cloud/learn/software-development-methodologies>
– https://www.tutorialspoint.com/software_engineering/software_development_models.htm
– <https://www.geeksforgeeks.org/software-engineering-development-models/>
Ces ressources présentent les modèles classiques comme le cycle en V, la méthode en cascade, RUP, Scrum, XP, et leurs avantages selon le contexte projet.

Le processus s'articule autour de trois phases principales :

1. Branche fonctionnelle

- **Capture des besoins fonctionnels** : production d'un modèle des besoins centrée sur le métier, afin de réduire le risque de développer un système inadapté. La maîtrise d'œuvre consolide et valide les spécifications.
- **Analyse** : étude approfondie de la spécification fonctionnelle pour définir clairement les fonctionnalités attendues, indépendamment de toute technologie.

2. Branche technique

- **Capture des besoins techniques** : recensement des contraintes et choix dimensionnant l'architecture (outils, matériels, intégration avec l'existant).
- **Conception générique** : définition des composants de l'architecture technique, indépendante des aspects fonctionnels. L'objectif est de réutiliser des mécanismes communs et de minimiser les risques techniques. Un prototype peut être recommandé pour valider l'architecture.

3. Phase de réalisation

- **Conception préliminaire** : intégration du modèle d'analyse dans l'architecture technique pour cartographier les composants.
- **Conception détaillée** : étude approfondie de la réalisation de chaque composant.
- **Codage** : développement et tests unitaires des composants.
- **Recette** : validation des fonctions du système complet.

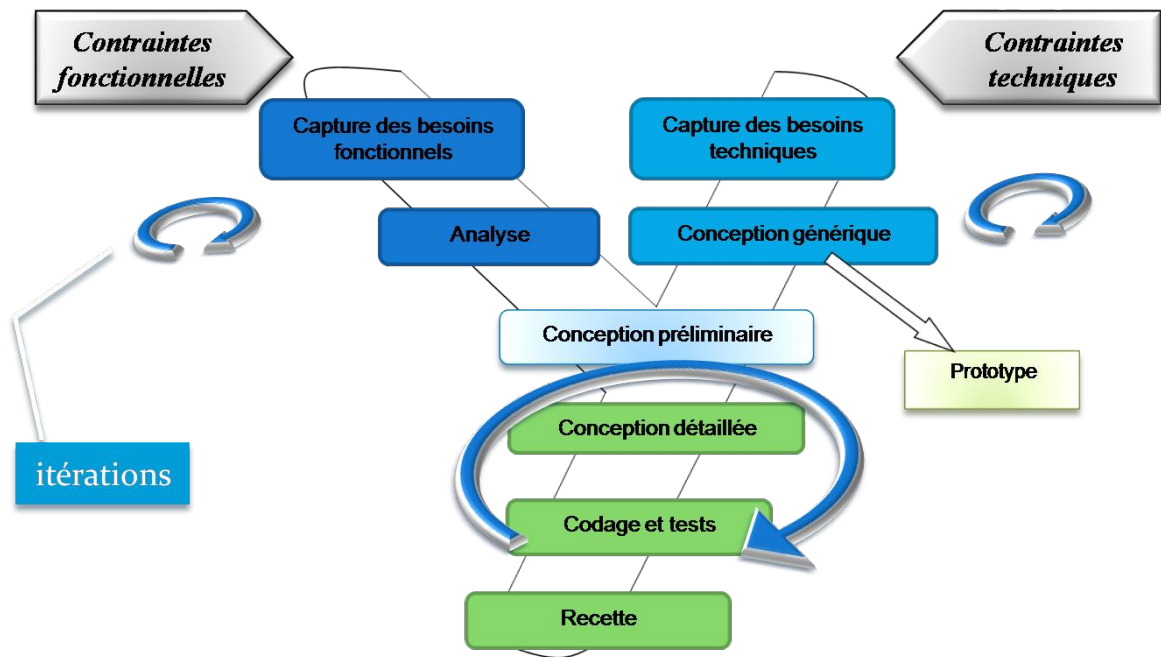


FIGURE 2.1 – Schéma présentant le processus 2TUP

Le processus 2TUP s'appuie sur UML tout au long du cycle de développement, car les différents diagrammes de ce dernier permettent en plus de leur facilité et de leur clarté, de bien modéliser le système à chaque étape.

b. Langage de modélisation

Dans le cadre du développement d'une application, il est essentiel de commencer par organiser et documenter les idées avant d'entamer la phase de réalisation. Cette démarche, appelée modélisation, consiste à définir les modules et les étapes du projet afin de garantir une conception claire et structurée. Pour mener à bien cette modélisation, nous avons choisi d'utiliser le langage UML, pour plusieurs raisons :

- **Un standard universel** : UML est la référence en matière de modélisation objet, reconnue et utilisée à l'échelle internationale.
- **Un langage visuel** : grâce à sa notation graphique, il permet de représenter visuellement des solutions objets, facilitant ainsi leur comparaison et leur évaluation.
- **Un langage formel et normalisé** : il apporte rigueur, précision et stabilité aux représentations.
- **Un outil de formalisation** : UML permet de produire et de structurer tous les documents techniques du projet, tout en affinant progressivement les détails de l'analyse au fur et à

mesure de son avancement.

- **Une continuité dans le cycle de développement** : il peut être utilisé depuis la capture des besoins utilisateurs jusqu'à la génération de tout ou partie du code.
- **Un support de communication efficace** : il fournit un cadre clair pour l'analyse et favorise la compréhension des représentations abstraites et complexes par l'ensemble des acteurs du projet.

3. Groupe de Travail

La réalisation de ce projet repose sur la contribution de plusieurs acteurs, regroupés en trois (03) catégories travaillant en synergie : le groupe de pilotage, le groupe de projet et le groupe des utilisateurs.

a. Groupe de pilotage

Ce groupe constitue l'instance dirigeante du projet. Il veille au bon déroulement des activités, planifie les échéances clés, examine les propositions issues du groupe de projet et prend les décisions stratégiques. Il se compose de :

- M. Innocent VALEA, Responsable Qualité de l'URCN, maître de stage
- Dr Yacouba OUATTARA, Maître Assistant en Informatique à l'IBAM, directeur de rapport

b. Groupe de projet

Il s'agit du groupe opérationnel chargé de la réalisation technique et organisationnelle du projet. Il joue un rôle d'interface entre le groupe de pilotage et les utilisateurs. Ce groupe est constitué principalement de Armand Hendrix OUEDRAOGO, étudiant en troisième (3^e) année MIAGE à l'IBAM

c. Groupe des utilisateurs

Ce groupe regroupe l'ensemble des futurs utilisateurs du système. Il intervient particulièrement dans la capture des besoins fonctionnels afin de garantir que la solution proposée réponde aux attentes réelles. Il est composé principalement du personnel de l'URCN.

4. Planning de réalisation

Afin d'assurer le respect des délais et la bonne exécution du projet, celui-ci a été découpé en différentes tâches planifiées selon un ordre logique. Pour ce faire, nous avons utilisé le diagramme de Gantt, un outil de planification et de gestion de projet permettant de visualiser dans le temps l'enchaînement et la durée des différentes étapes. Le planning prévisionnel du projet est synthétisé dans la figure 2.2 :

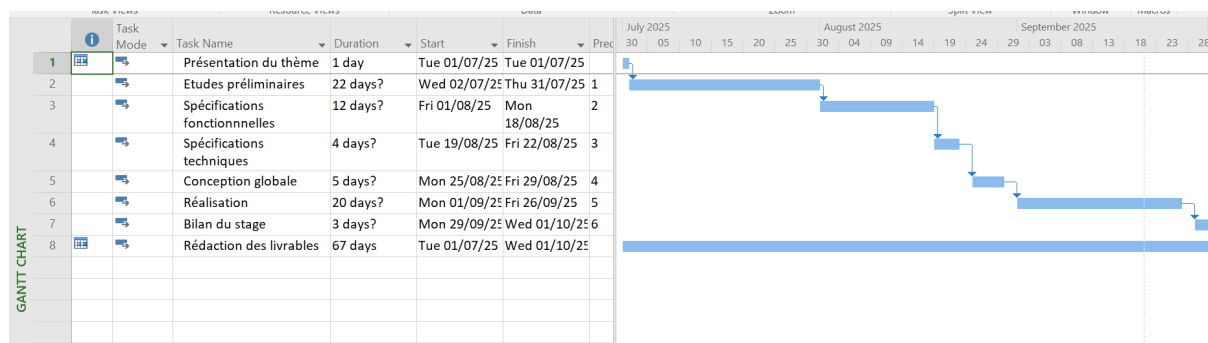


FIGURE 2.2 – Planning prévisionnel de réalisation du projet

La figure 2.2 illustre le planning de réalisation de notre stage, qui a débuté le 1 juillet 2025 et s'est achevé le 30 septembre 2025. Au cours de cette période, plusieurs tâches ont été menées, notamment : la présentation du thème, l'étude préliminaire, l'identification des spécifications fonctionnelles, l'identification des spécifications techniques, la conception globale, la réalisation, le bilan et l'élaboration des livrables. Après la présentation du planning de réalisation, nous abordons maintenant l'expression des besoins, qui met en évidence les acteurs impliqués ainsi que les échanges de messages entre eux et le système de gestion des expressions de besoins de l'URCN.

II. Expressions des besoins

Cette partie a pour objectif, d'une part, de présenter l'étude de l'existant, afin de mettre en évidence les limites et insuffisances du système actuel de gestion des expressions de besoins au sein de l'URCN. D'autre part, elle consiste à décrire le fonctionnement attendu du futur système, en mettant en avant les améliorations envisagées pour répondre efficacement aux besoins identifiés. Enfin, cette section permettra de faire ressortir, de manière structurée, les spécifications fonctionnelles et les spécifications techniques qui serviront de base à la conception et au développement du module intégré dans l'ERP Dolibarr.

1. Etude de l'existant

Avant de proposer une solution adaptée, il est indispensable de réaliser une étude du système existant. Cette étape permet d'identifier ses limites ainsi que de mettre en évidence les besoins fonctionnels auxquels le futur module devra répondre.

a. Constat du système actuel

Lors de notre analyse du système existant l'URCN pour la gestion des expressions de besoin, nous avons constaté que le processus repose encore largement sur une gestion manuelle, bien que certains outils numériques soient utilisés ponctuellement.

Fonctionnement actuel

- Toute personne souhaitant soumettre une expression de besoin doit remplir un formulaire PDF éditable (voir Annexe 7).
- Ce formulaire contient des champs pour :
 - Le nom et prénom du demandeur
 - Le projet ou le service concerné
 - L'imputation (si connue)
 - La date du jour
 - Un tableau listant les items demandés : désignation, référence, quantité
 - Trois fournisseurs recommandés, pour l'ensemble de l'expression
- Une fois le formulaire rempli :
 - Il est transmis au coordonnateur de projet (si lié à un projet) ou au chef de service (si lié à un service)
 - Ces premiers validateurs analysent les éléments et signent électroniquement via OpenSign (voir Annexe 6)
 - OpenSign envoie automatiquement une demande de signature au second validateur (gestionnaire du projet ou du service)
 - Ce dernier signe également via OpenSign, puis télécharge le document
- Le document signé est ensuite transmis au service achat et approvisionnement, où les responsables achat prennent en charge le traitement :
 - Ils remplissent des champs complémentaires sur le formulaire :
 - Date de réception
 - Numéro EB
 - Numéro de document

- Date de transmission
 - Ces informations sont obtenues via Dolibarr, lors de l'établissement d'une demande de cotation liée à l'expression
 - Chaque fournisseur retenu reçoit un numéro de document distinct, et le numéro EB est unique pour chaque expression
 - Les responsables achat peuvent ajuster certains noms d'items pour corriger les incohérences ou imprécisions du demandeur
 - Leur signature est intégrée directement dans Dolibarr, de manière numérique : le système récupère leur identifiant et l'associe à l'expression traitée

Failles et limites du système actuel

Malgré une certaine structuration, ce système présente plusieurs failles :

- **Risques de modification non contrôlée** : le formulaire PDF reste éditable, ce qui peut compromettre l'intégrité des données
- **Absence de traçabilité centralisée** : les validations et traitements ne sont pas sauvegardés dans un système unique, rendant difficile le suivi des étapes
- **Dépendance à des outils externes** : OpenSign fonctionne en parallèle de Dolibarr, sans intégration directe, ce qui complexifie le workflow
- **Manque de contrôle sur les fournisseurs** : les recommandations ne sont pas liées aux items, et les décisions finales sont ajustées manuellement
- **Pas de vérification automatique** : aucune validation technique ou logique n'est appliquée aux données saisies (quantité, référence, cohérence)
- **Sécurité limitée** : le document peut être modifié après signature, sauf pour les champs signés, ce qui ouvre la porte à des altérations involontaires ou malveillantes

b. Exigences fonctionnelles identifiées

Face à ces constats, nous avons défini un ensemble d'exigences fonctionnelles pour guider la conception du nouveau module. Ces exigences sont résumées dans le tableau suivant :

TABLE 2.2 – Exigences fonctionnelles du module de gestion des expressions de besoin

Code exigence	Intitulé	Description
EX1	Création valide	L'utilisateur peut créer une expression de besoin avec des données complètes.
EX2	Validation des données	Le système empêche la soumission ou le traitement si les données sont incomplètes ou invalides.
EX3	Validation par le validateur	Un utilisateur ayant le rôle de validateur peut approuver une expression selon son niveau.
EX4	Changement de statut	Le statut de l'expression évolue correctement selon les actions effectuées (soumission, validation, traitement).
EX5	Droits d'accès	Les utilisateurs accèdent uniquement aux expressions qui leur sont destinées selon leur rôle (demandeur, validateur, responsable).
EX6	Notification métier	Le système déclenche une notification vers les utilisateurs concernés à chaque étape clé (soumission, validation, traitement), selon leur rôle.

Maintenant que nous avons vu la liste des exigences, passons à la description du fonctionnement attendu de l'application.

2. Description du fonctionnement attendu de l'application

Le nouveau système vise à permettre à l'URCN de gérer efficacement les expressions de besoins depuis leur création jusqu'à leur traitement final. Après développement, l'application offrira un ensemble de fonctionnalités organisées autour d'un flux de travail clair et automatisé.

a. *Création et soumission des expressions de besoins*

L'utilisateur (demandeur) pourra créer une expression de besoin, l'enregistrer en brouillon pour modification ultérieure ou la soumettre pour validation.

Lorsqu'une expression est soumise :

- Elle est transmise en premier lieu soit au coordonnateur de projet, soit au chef de service, selon que la demande concerne un projet ou un service.
- Le valideur concerné reçoit une notification lui indiquant qu'une nouvelle expression de besoin est en attente de validation.

b. Validation par le chef de service ou le coordonnateur

Le valideur peut consulter les détails de l'expression reçue et décider :

- D'accepter tous les items et signer
- De rejeter certains items et signer
- De rejeter tous les items

Dans tous les cas, sa signature électronique constitue une preuve formelle qu'il a examiné la demande et exprimé son avis.

Après signature, si au moins un item a été accepté :

- Le demandeur est notifié pour connaître l'état d'avancement de sa demande
- Le gestionnaire reçoit également une notification l'informant qu'une expression est prête pour validation de sa part

c. Validation par le gestionnaire

Le gestionnaire applique le même principe que le chef de service ou le coordonnateur :

- Accepter, rejeter ou valider partiellement les items

Les items validés passent ensuite en phase de traitement. Le demandeur est tenu informé par notification.

d. Traitement par les responsables des achats

Les responsables achats prennent en charge les items validés. Ils disposent de deux options :

- Enregistrer en brouillon pour modification ultérieure
- Signer pour marquer la fin du traitement (après signature, aucune modification n'est possible)

Après finalisation du traitement :

- Le demandeur reçoit une notification confirmant la clôture de son expression
- Les responsables achats peuvent également échanger des messages liés à une expression donnée afin de tenir informé le demandeur de la disponibilité ou de l'évolution de certains items

e. Génération automatique de documents et gestion des pièces jointes

- À chaque traitement final, un document PDF est généré automatiquement. Ce document contient le récapitulatif des éléments demandés ainsi qu'un QR code garantissant l'authenticité des informations.
- Une section fichiers joints est intégrée, permettant au demandeur, aux validateurs ou aux responsables achats d'ajouter des fichiers complémentaires (ex. fiche technique, devis, justificatif), afin de faciliter la prise de décision.

f. Consultation des statistiques

Le système intègre un module de visualisation des statistiques relatives aux expressions de besoins traitées.

- L'accès à cette section est réservé à une catégorie d'utilisateurs disposant des droits adéquats.
- Les statistiques peuvent être exportées dans plusieurs formats (PDF, Excel, CSV).

Avantages du nouveau système

La mise en place de ce module de gestion des expressions de besoins présente plusieurs avantages pour l'URCN :

- **Efficacité accrue** : réduction des délais de traitement grâce à l'automatisation des flux de validation et de notification.
- **Traçabilité et transparence** : chaque étape (création, validation, traitement) est enregistrée et documentée, assurant un suivi clair.
- **Sécurité renforcée** : les signatures électroniques et l'intégration d'un QR code garantissent l'authenticité des documents générés.
- **Flexibilité et précision** : possibilité de valider partiellement une expression en acceptant certains items et en rejetant d'autres.
- **Suivi de l'évolution des expressions de besoins** : grâce à un système de notifications, le créateur ainsi que les validateurs sont informés en temps réel de chaque mise à jour.
- **Accessibilité et centralisation** : toutes les informations et pièces jointes liées aux expressions de besoins sont centralisées et accessibles via Dolibarr.
- **Aide à la décision** : module statistique permettant de générer des rapports détaillés et exploitables pour le pilotage stratégique.

Après avoir présenté les principales fonctionnalités et le fonctionnement général du nouveau système, il convient désormais de détailler les spécifications techniques qui encadrent sa

conception et garantissent sa conformité aux exigences du projet.

3. Spécifications fonctionnelles

La spécification fonctionnelle constitue une étape clé dans le processus de développement, car elle décrit de manière précise les fonctionnalités attendues du logiciel en vue de sa réalisation. Elle permet de formaliser les exigences du futur système en tenant compte des besoins exprimés par les utilisateurs.

Dans cette section, nous détaillerons les différentes exigences fonctionnelles à travers :

- L'identification des acteurs qui interagiront avec le système
- La définition des cas d'utilisation illustrant les scénarios d'interaction
- La présentation des diagrammes permettant de modéliser le comportement et la structure du système

Cette démarche vise à garantir une compréhension partagée du fonctionnement attendu du logiciel, tout en facilitant sa conception et son implémentation.

a. Identification des acteurs

Un acteur définit un ensemble cohérent de rôles qu'un utilisateur ou une entité externe peut jouer en interagissant avec le système. Dans notre cas, il s'agit de :

- **Utilisateur** : celui qui crée l'expression de besoin
- **Administrateur** : celui qui a la possibilité de voir les statistiques des expressions
- **Valideur1** : celui qui est chargé de faire la validation de l'expression au premier niveau il est un coordonnateur de projet ou un chef de service
- **Valideur2** : celui qui est chargé de faire la validation de l'expression au deuxième niveau il est un gestionnaire de service ou de projet
- **ResponsableAchat** : celui chargé de traiter l'expression après validation il est forcément du service achat et approvisionnement
- **Système** : C'est le logiciel existant destiné à être utilisé au sein de l'urcn

b. Identification des cas d'utilisation

Les cas d'utilisation désignent l'ensemble des interactions qui vont permettre à l'acteur d'atteindre son objectif en utilisant le système. Le tableau 2.3 liste les cas d'utilisation de notre

futur système.

TABLE 2.3 – Liste des cas d'utilisation du système

Numéro	Cas d'utilisation	Description	Acteurs
CU01	S'authentifier	Système d'authentification de base de Dolibarr	Tous les acteurs
CU02	Gérer expression	Enregistrer, soumettre, voir, modifier, supprimer une expression	Utilisateur
CU03	Gérer fichiers joints	Ajouter, télécharger, modifier, supprimer un fichier joint à une expression	Tous les acteurs
CU04	Voir expressions à statuer premier niveau	Voir les expressions soumises pour validation	Valideur1
CU05	Statuer expression premier niveau	Accepter ou rejeter une expression au premier niveau	Valideur1
CU06	Voir expressions à statuer deuxième niveau	Voir les expressions acceptées au premier niveau en attente de validation	Valideur2
CU07	Statuer expression deuxième niveau	Accepter ou rejeter une expression au deuxième niveau	Valideur2
CU08	Voir expressions à traiter	Voir les expressions à traiter	ResponsableAchat
CU09	Traiter expression	Traiter une expression	ResponsableAchat
CU10	Voir statistiques	Voir les statistiques concernant les expressions	Administrateur
CU11	Exporter données statistiques	Exporter les données des statistiques en plusieurs formats	Administrateur
CU12	Notifier utilisateur	Envoyer une notification au demandeur après validation ou traitement	Valideur1, Valideur2, ResponsableAchat
CU13	Notifier valideur1	Notifier le premier valideur lorsqu'une expression est soumise	Utilisateur

Suite à la page suivante...

Numéro	Cas d'utilisation	Description	Acteurs
CU14	Notifier validateur2	Notifier le deuxième validateur lorsqu'une expression passe au deuxième niveau	Validateur1
CU15	Notifier responsable achat	Notifier les responsables achat lorsqu'une expression est prête à être traitée	Validateur2
CU16	Voir notifications	Lire les notifications reçues	Utilisateur
CU17	Générer PDF	Générer un PDF à la fin du traitement de l'expression	ResponsableAchat
CU18	Envoyer notification item	Envoyer des messages au demandeur concernant un item d'une expression	ResponsableAchat
CU19	Fournir utilisateur	Donner les informations des utilisateurs du système existant	Systeme

Pour une meilleure visualisation globale de l'ensemble des cas d'utilisation du système, nous présentons le diagramme des cas d'utilisation dans la suite de notre travail.

c. Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation est une modélisation UML ((voir Annexe 1)) permettant de représenter globalement le comportement fonctionnel d'un système logiciel. Chaque cas d'utilisation décrit une interaction précise entre le système et un utilisateur, qu'il s'agisse d'une personne ou d'un dispositif technique. Ce type de diagramme contribue à structurer les besoins exprimés, à clarifier les attentes fonctionnelles et à identifier les acteurs ainsi que les fonctionnalités du système. La figure 2.3 illustre cette représentation.

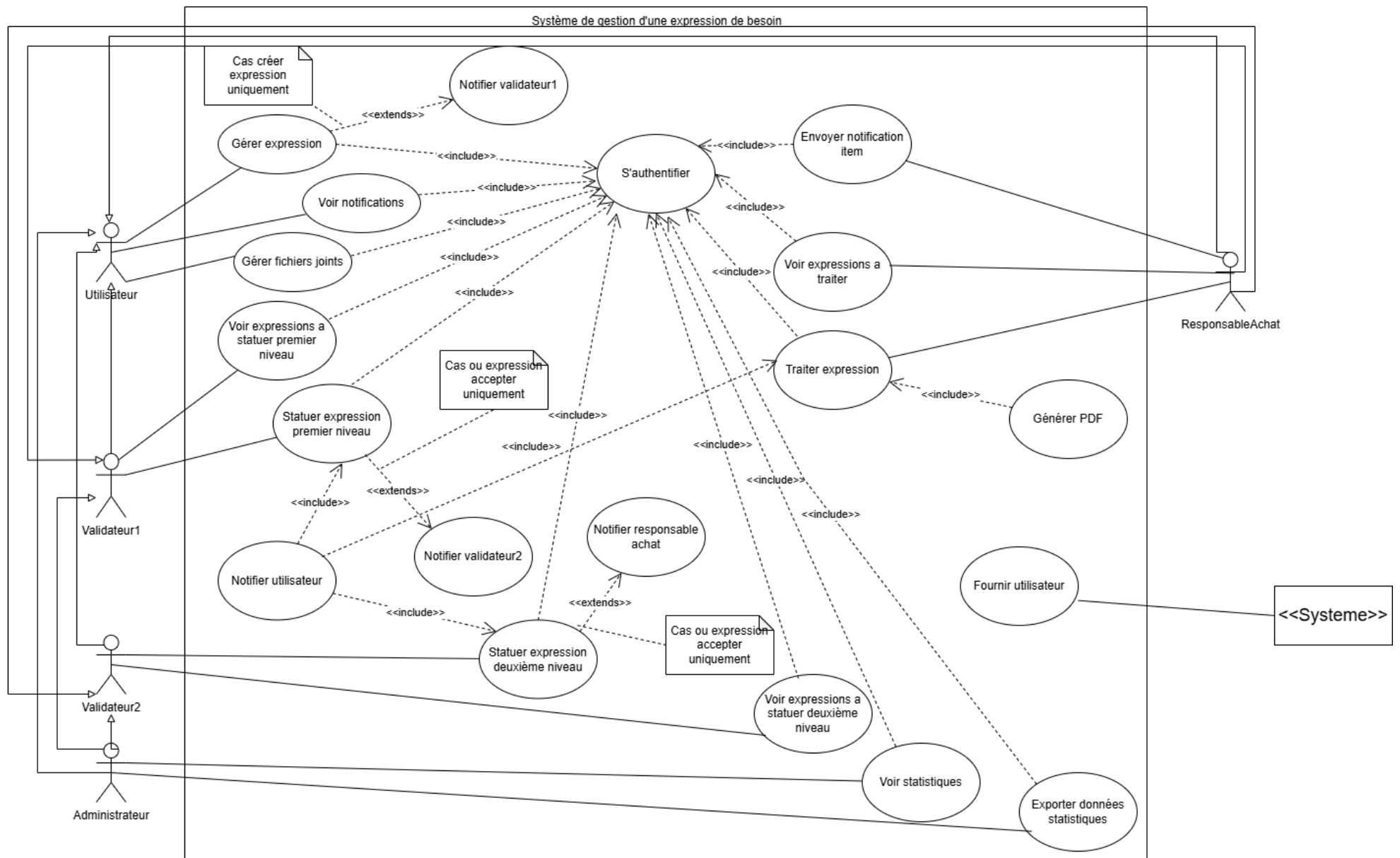


FIGURE 2.3 – Diagramme des cas d'utilisation

Pour mieux comprendre les cas d'utilisation illustrés dans le diagramme des cas d'utilisation, nous présentons à titre d'exemple les descriptions textuelles de quelques cas d'utilisation.

d. Description textuelle de certains cas d'utilisation

Les cas d'utilisation offrent une structure organisée pour comprendre les besoins des utilisateurs ainsi que les objectifs du système. Alors que le diagramme des cas d'utilisation met en avant les principales fonctionnalités du système selon le point de vue des acteurs, la description textuelle des cas d'utilisation permet une exposition détaillée des interactions entre les acteurs et les cas d'utilisation. Dans cette optique, nous nous focaliserons sur la description approfondie des cas d'utilisation suivant : S'authentifier, Créer expression, Statuer expression premier niveau, Statuer expression deuxième, Traiter expression.

CU01 : S'authentifier

Le résumé de ce cas d'utilisation se trouve dans le tableau 2.4 .

TABLE 2.4 – Description du CU01 « S'authentifier »

Élément	Contenu
Résumé	Ce cas permet à l'utilisateur de s'authentifier
Acteurs	Tous les acteurs
Précondition	Néant
Scénario nominal	1. L'acteur demande à se connecter 2. Le système affiche le formulaire de connexion 3. L'acteur remplit le formulaire et le soumet 4. Le système vérifie les données saisies [A] 5. Le système redirige l'acteur vers la page d'accueil
Scénario alternatif	[A] : Identifiants ou mot de passe incorrect 1. Le système informe l'acteur de l'échec de la connexion 2. Le système renvoie le formulaire de connexion et le scénario reprend à partir du point 3 du scénario nominal
Post-condition	L'acteur accède à la page d'accueil

Le tableau 2.4 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « S'authentifier ». D'abord, l'utilisateur demande à se connecter, puis il reçoit un formulaire de connexion qu'il doit remplir en fournissant son nom d'utilisateur et son mot de passe. Une fois soumise, le système vérifie les informations saisies. Si les informations sont incorrectes le

système lui informe que ses identifiants sont incorrects et renvoie le même formulaire pour qu'il réessaie. Au cas où les informations sont correctes, l'utilisateur est dirigé vers la page d'accueil de dolibarr. La procédure est valable pour tous les acteurs du système.

CU02 : Créer une expression

Le résumé de ce cas d'utilisation se trouve dans le tableau 2.5

TABLE 2.5 – Description du CU02 « Créer une expression »

Élément	Contenu
Résumé	Ce cas permet à l'utilisateur de créer une expression
Acteurs	Tous les acteurs
Précondition	Être connecté et module d'expression de besoin activé
Scénario nominal	1. L'acteur accède au module et clique sur « Nouvelle expression » 2. Le système affiche la page de création 3. L'acteur remplit le formulaire et l'enregistre en brouillon ou la soumet 4. Le système vérifie les données saisies [A] 5. Le système affiche un message de succès et redirige vers la page des détails de l'expression
Scénario alternatif	[A] : Données non valides 1. Le système informe l'acteur de l'erreur 2. Le système renvoie le formulaire et le scénario reprend au point 3 du scénario nominal
Post-condition	L'acteur accède à la page des détails de l'expression

Le tableau 2.5 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « Créer une expression ». L'utilisateur doit d'abord être connecté. Une fois la connexion établie, il accède au module d'expression de besoin et clique sur le menu « Nouvelle expression ». Il reçoit alors un formulaire de création d'expression qu'il doit remplir en renseignant tous les champs demandés. Une fois le formulaire complété, il enregistre en brouillon ou soumet l'expression au système, qui procède à la vérification des informations saisies. Si celles-ci ne sont pas valides, le système retourne un message d'erreur et renvoie le même formulaire de création afin que l'utilisateur puisse réessayer. En revanche, si les informations sont correctes, l'utilisateur reçoit un message de succès et est redirigé vers la page des détails de l'expression. Cette procédure est applicable à tous les acteurs du système.

CU03 : Statuer expression premier niveau

Le résumé de ce cas d'utilisation se trouve dans le tableau 2.6

TABLE 2.6 – Description du CU03 « Statuer expression premier niveau »

Élément	Contenu
Résumé	Ce cas permet au premier validateur d'accepter ou rejeter une expression
Acteurs	Validateur1
Précondition	Être connecté et module d'expression de besoin activé
Scénario nominal	1. L'acteur clique sur « Liste des expressions à valider » 2. Le système affiche la liste 3. L'acteur choisit une expression et clique sur « Passer à la validation » 4. Le système affiche les détails et les champs à compléter 5. L'acteur donne son avis et clique sur « Signer » 6. Le système vérifie les données saisies [A] 7. Le système affiche un message de succès et redirige vers la liste
Scénario alternatif	[A] : Données non valides 1. Le système informe l'acteur de l'erreur 2. Le système renvoie la page de validation et le scénario reprend au point 5 du scénario nominal
Post-condition	L'acteur retourne sur la page « Liste des expressions à valider »

Le tableau 2.6 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « Statuer expression premier niveau ». L'utilisateur doit d'abord être connecté. Une fois la connexion établie, il accède au module d'expression de besoin et clique sur le menu « Liste des expressions à valider ». Il reçoit alors la liste des expressions qu'il est censé valider. Il choisit parmi cette liste l'expression qu'il souhaite valider et clique sur le bouton « Passez à la validation ». Dès lors, il est redirigé vers la page de validation de l'expression en question, où il peut consulter les détails de l'expression et remplir des champs. L'acteur donne son avis sur chacun des items de l'expression concernée en remplissant les champs, puis clique sur le bouton « Signer ». Si les informations saisies lors de l'avis ne sont pas valides, le système retourne un message d'erreur et renvoie la page de validation de l'expression afin que l'utilisateur puisse réessayer. En revanche, si les informations sont correctes, l'utilisateur reçoit un message de succès et est redirigé vers la page « Liste des expressions à valider ». Cette procédure est applicable uniquement à l'acteur « Validateur1 ».

CU04 : Statuer expression deuxième niveau

Le résumé de ce cas d'utilisation se trouve dans le tableau 2.7.

TABLE 2.7 – Description du CU04 « Statuer expression deuxième niveau »

Élément	Contenu
Résumé	Ce cas permet au deuxième validateur d'accepter ou rejeter une expression
Acteurs	Validateur2
Précondition	Être connecté et module d'expression de besoin activé
Scénario nominal	1. L'acteur clique sur « Liste des expressions à valider » 2. Le système affiche la liste 3. L'acteur choisit une expression et clique sur « Passer à la validation » 4. Le système affiche les détails et les champs à compléter 5. L'acteur donne son avis et clique sur « Signer » 6. Le système vérifie les données saisies [A] 7. Le système affiche un message de succès et redirige vers la liste
Scénario alternatif	[A] : Données non valides 1. Le système informe l'acteur de l'erreur 2. Le système renvoie la page de validation et le scénario reprend au point 5 du scénario nominal
Post-condition	L'acteur retourne sur la page « Liste des expressions à valider »

Le tableau 2.7 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « Statuer expression premier niveau ». L'utilisateur doit d'abord être connecté. Une fois la connexion établie, il accède au module d'expression de besoin et clique sur le menu « Liste des expressions à valider ». Il reçoit alors la liste des expressions qu'il est censé valider. Il choisit parmi cette liste l'expression qu'il souhaite valider et clique sur le bouton « Passez à la validation ». Dès lors, il est redirigé vers la page de validation de l'expression en question, où il peut consulter les détails de l'expression et remplir des champs. L'acteur donne son avis sur chacun des items de l'expression concernée en remplissant les champs, puis clique sur le bouton « Signer ». Si les informations saisies lors de l'avis ne sont pas valides, le système retourne un message d'erreur et renvoie la page de validation de l'expression afin que l'utilisateur puisse réessayer. En revanche, si les informations sont correctes, l'utilisateur reçoit un message de succès et est redirigé vers la page « Liste des expressions à valider ». Cette procédure est applicable uniquement à l'acteur « Validateur2 ».

CU05 : Traiter une expression

Le résumé de ce cas d'utilisation se trouve dans le tableau 2.8.

TABLE 2.8 – Description du CU05 « Traiter une expression »

Élément	Contenu
Résumé	Ce cas permet au responsable achat de traiter une expression
Acteurs	ResponsableAchat
Précondition	Être connecté et module d'expression de besoin activé
Scénario nominal	1. L'acteur clique sur « Liste des expressions à traiter » 2. Le système affiche la liste 3. L'acteur choisit une expression et clique sur « Passer au traitement » 4. Le système affiche les détails et les champs à compléter 5. L'acteur remplit les champs et clique sur « Signer » 6. Le système vérifie les données saisies [A] 7. Le système affiche un message de succès et redirige vers la liste
Scénario alternatif	[A] : Données non valides 1. Le système informe l'acteur de l'erreur 2. Le système renvoie la page de traitement et le scénario reprend au point 5 du scénario nominal
Post-condition	L'acteur retourne sur la page « Liste des expressions à traiter »

Le tableau 2.8 décrit les interactions entre les acteurs et le système pour le cas d'utilisation « Traiter une expression ». L'utilisateur doit d'abord être connecté. Une fois la connexion établie, il accède au module d'expression de besoin et clique sur le menu « Liste des expressions à traiter ». Il reçoit alors la liste des expressions qu'il est censé traiter. Il choisit parmi cette liste l'expression qu'il souhaite traiter et clique sur le bouton « Passez au traitement ». Dès lors, il est redirigé vers la page de traitement de l'expression en question, où il peut consulter les détails de l'expression et remplir des champs. L'acteur remplit les champs demandés, puis clique sur le bouton « Signer ». Si les informations saisies ne sont pas valides, le système retourne un message d'erreur et renvoie la page de traitement de l'expression afin que l'utilisateur puisse réessayer. En revanche, si les informations sont correctes, l'utilisateur reçoit un message de succès et est redirigé vers la page « Liste des expressions à traiter ». Cette procédure est applicable uniquement à l'acteur « ResponsableAchat ». Après avoir vu la description passons maintenant aux interactions entre les acteurs et le système à travers les diagrammes de séquence, afin d'illustrer le déroulement logique des opérations et la dynamique des échanges.

e. Diagramme de séquence

Les diagrammes de séquence illustrent les échanges entre le système et les utilisateurs en exposant, sous forme de scénarios, la séquence temporelle des messages émis à partir d'un cas d'utilisation donné. Dans cette section, les diagrammes de séquence portent principalement sur cinq aspects : S'authentifier, Créer une expression, Statuer expression premier niveau, Statuer expression deuxième niveau, Traiter expression.

À noter

L'interprétation des différents diagrammes de séquence suit le schéma suivant : les flèches dirigées vers la droite représentent les messages de demande envoyés par les acteurs, tandis que celles dirigées vers la gauche correspondent aux messages de réponse renvoyés par le système. Les lignes verticales traduisent les périodes d'activité des acteurs, et la lecture se fait de haut en bas.

CU01 : S'authentifier

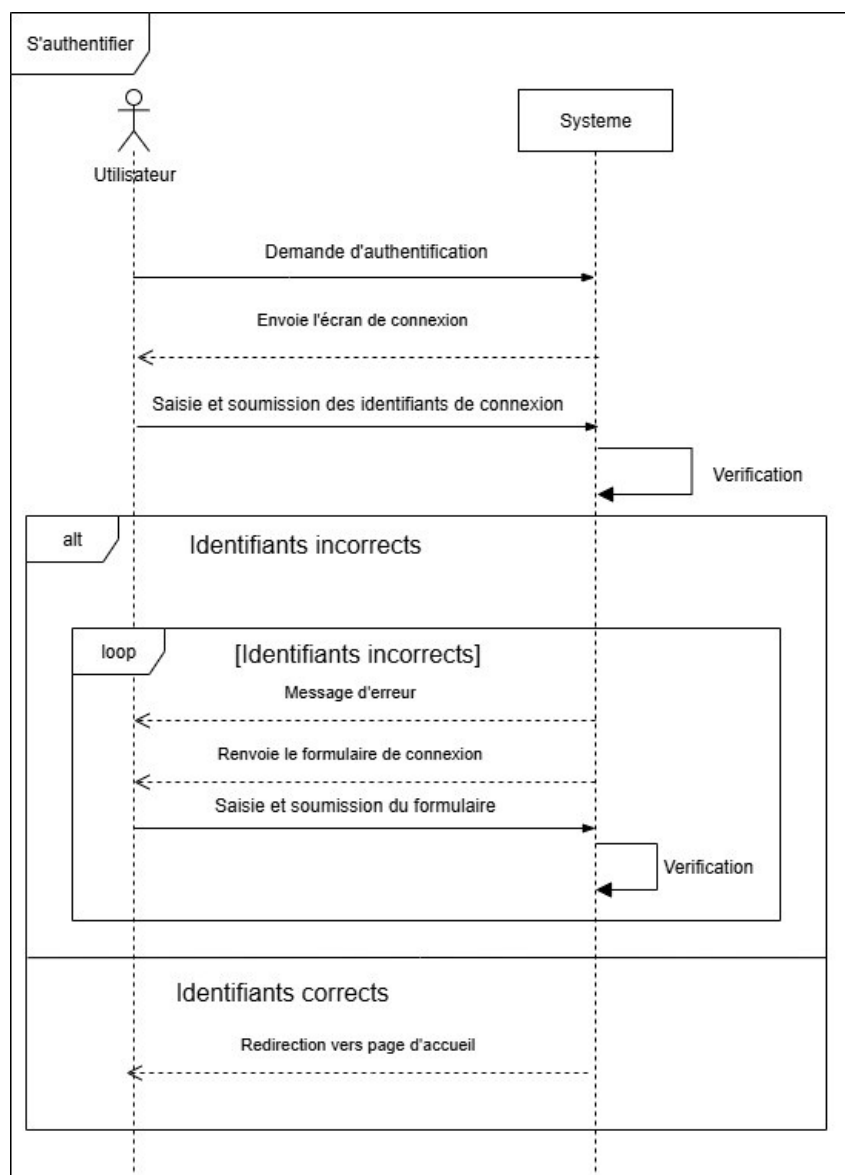


FIGURE 2.4 – Diagramme de séquence du cas d'utilisation « S'authentifier »

La figure 2.4 présente graphiquement les messages échangés au niveau du cas d'utilisation « S'authentifier ». L'utilisateur demande à se connecter au système, qui lui renvoie un formulaire pour saisir son nom d'utilisateur et son mot de passe pour se connecter. Les informations saisies sont vérifiées à deux (02) niveaux par le système. Il faut que les informations saisies respectent le format de base attendu pour les entrées. Si les normes ne sont pas respectées, l'utilisateur reçoit un message d'erreur avec le message lui indiquant qu'il ne respecte pas le bon format. Au cas où les informations saisies sont syntaxiquement valides, une deuxième vérification est faite concernant la compatibilité des informations dans le système. En cas d'informations incompatibles, l'utilisateur reçoit un message d'erreur indiquant que ses identifiants sont incorrects. Dans le cas contraire l'utilisateur s'est authentifié avec succès et accède à la page d'accueil de dolibarr.

CU02 : Créer une expression

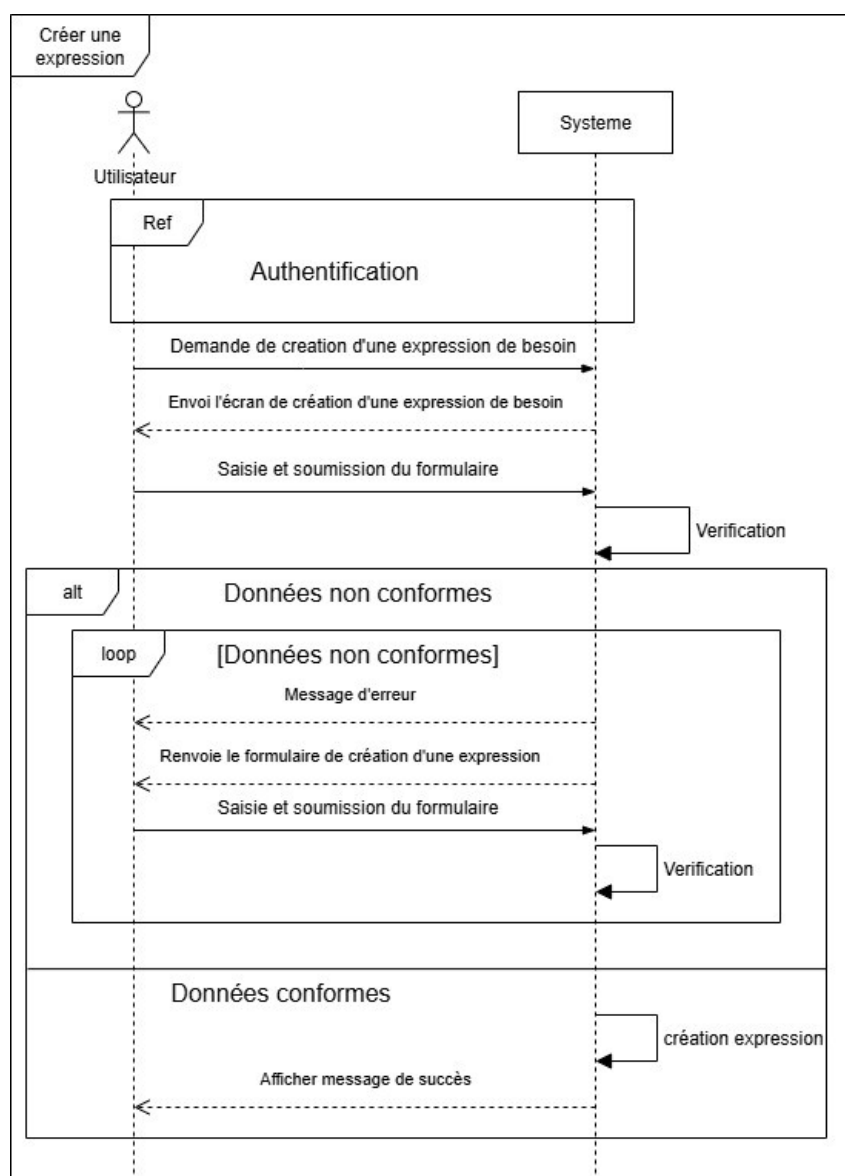


FIGURE 2.5 – Diagramme de séquence du cas d'utilisation « Créer une expression »

La figure 2.5 présente graphiquement les messages échangés au niveau du cas d'utilisation « Créer une expression ». L'utilisateur, après connexion, demande au système le formulaire de création d'une expression de besoin. Le système lui envoie le formulaire de création d'expression de besoin. L'utilisateur le remplit et soumet son expression. Les informations saisies sont ensuite vérifiées par le système. Il est impératif que ces informations respectent le format de base attendu pour les entrées. Si les normes ne sont pas respectées, l'utilisateur reçoit un message d'erreur lui indiquant que le format est incorrect, et il est invité à ressaisir les informations. En revanche, si les données saisies sont syntaxiquement valides, l'expression est créée et le système retourne un message de succès.

CU03 : Statuer expression premier niveau

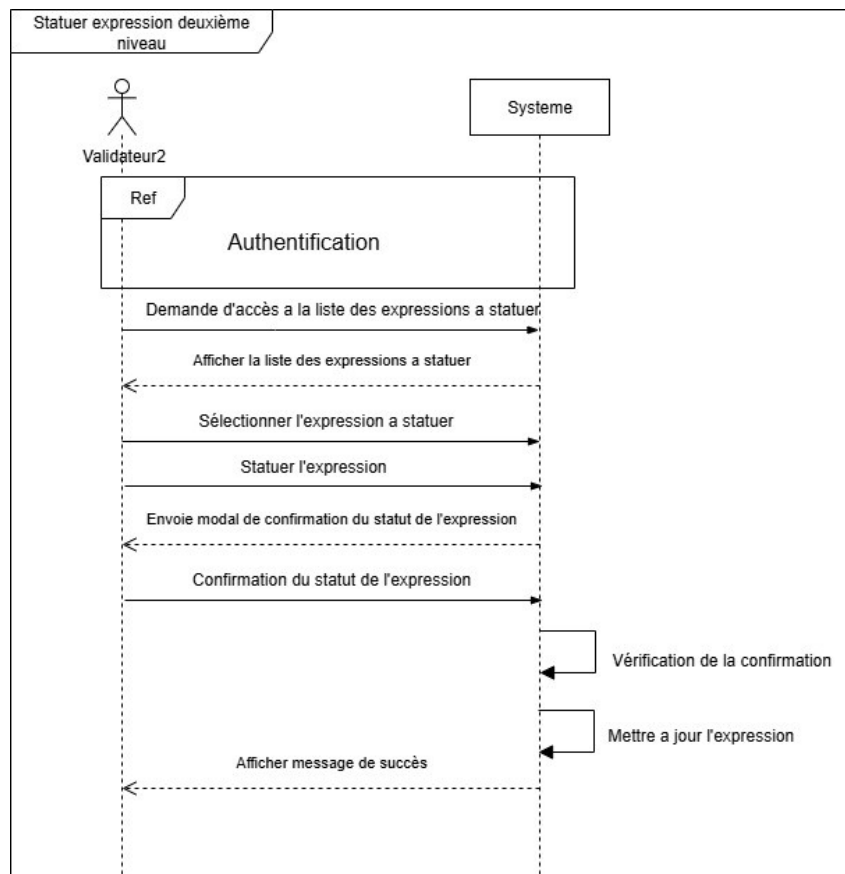


FIGURE 2.6 – Diagramme de séquence du cas d'utilisation « Statuer expression premier niveau »

La figure 2.6 présente graphiquement les messages échangés au niveau du cas d'utilisation « Statuer expression premier niveau ». Le validateur, après connexion, demande au système la liste des expressions à statuer. Le système lui envoie la liste des expressions à statuer. Le validateur statue l'expression en question en remplissant les champs demander et en soumettant. Les informations saisies sont ensuite vérifiées par le système. Il est impératif que ces informations respectent le format de base attendu pour les entrées. Si les normes ne sont pas respectées, l'utilisateur reçoit un message d'erreur lui indiquant que le format est incorrect, et il est invité à ressaisir les informations. En revanche, si les données saisies sont syntaxiquement valides le système demande à l'utilisateur une confirmation et après confirmation l'expression est statuée et un message de succès est envoyé à l'utilisateur.

CU04 : Statuer expression deuxième niveau

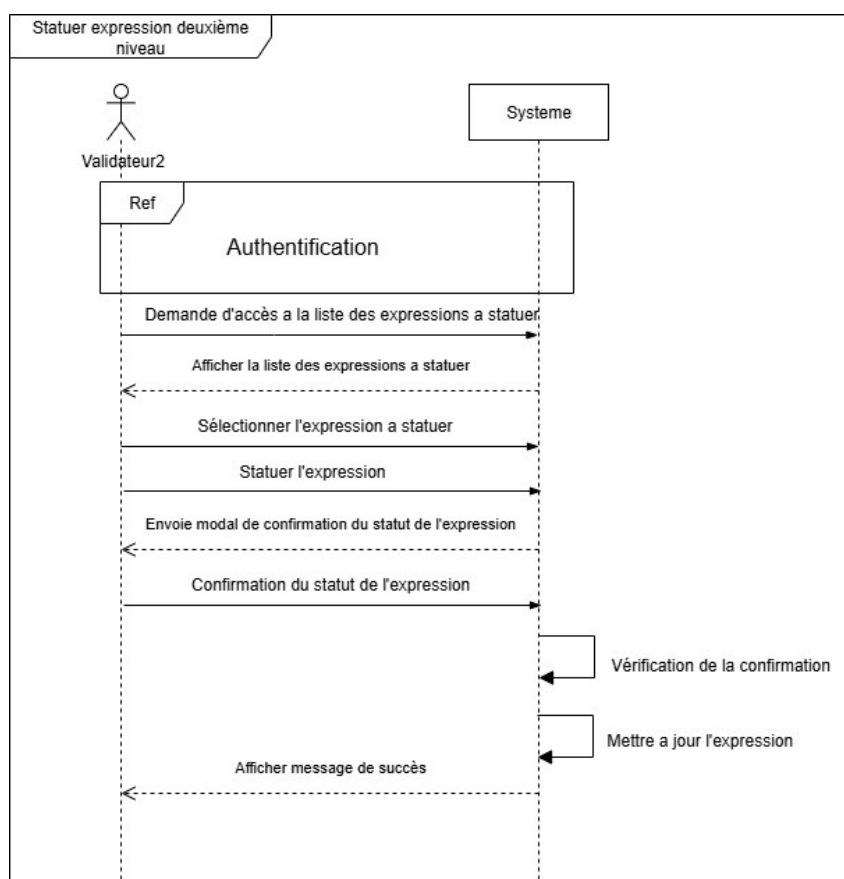


FIGURE 2.7 – Diagramme de séquence du cas d'utilisation « Statuer expression deuxième niveau »

La figure 2.7 présente graphiquement les messages échangés au niveau du cas d'utilisation « Statuer expression deuxième niveau ». Le valideur, après connexion, demande au système la liste des expressions à statuer. Le système lui envoie la liste des expressions à statuer. Le valideur statue l'expression en question en remplissant les champs demander et en soumettant. Les informations saisies sont ensuite vérifiées par le système. Il est impératif que ces informations respectent le format de base attendu pour les entrées. Si les normes ne sont pas respectées, l'utilisateur reçoit un message d'erreur lui indiquant que le format est incorrect, et il est invité à ressaisir les informations. En revanche, si les données saisies sont syntaxiquement valides le système demande à l'utilisateur une confirmation et après confirmation l'expression est statué et un message de succès est envoyé à l'utilisateur.

CU05 : Traiter une expression

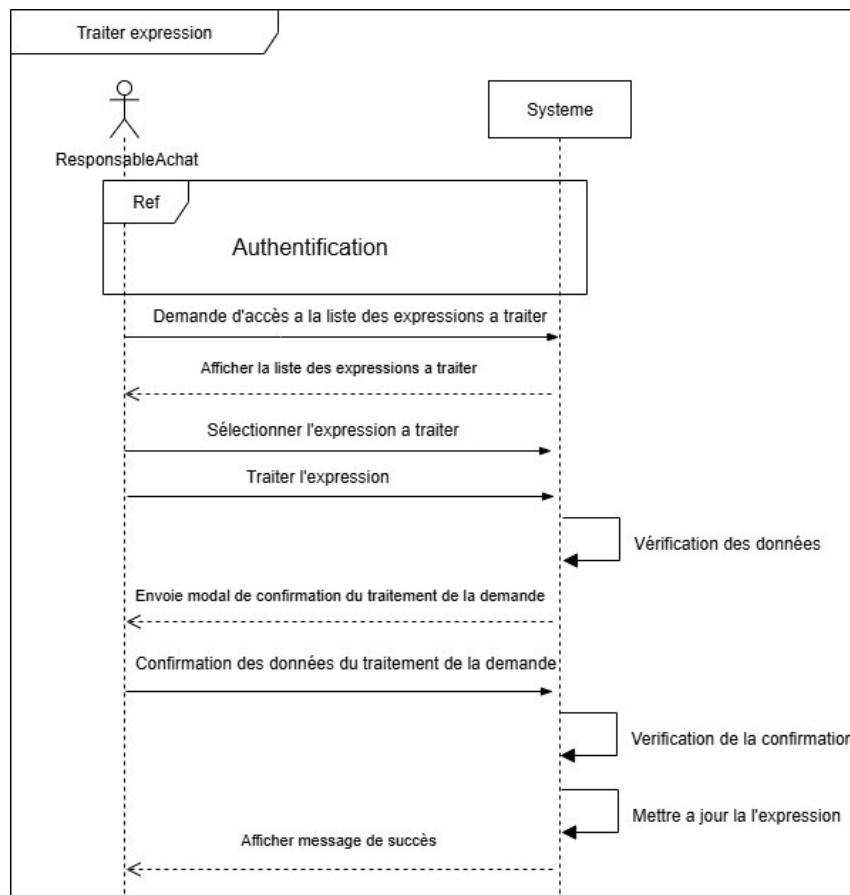


FIGURE 2.8 – Diagramme de séquence du cas d'utilisation « Traiter une expression »

La figure 2.8 présente graphiquement les messages échangés au niveau du cas d'utilisation « Traiter expression ». Le responsable achat, après connexion, demande au système la liste des expressions à traiter. Le système lui envoie la liste des expressions à traiter. Le responsable achat traite l'expression en question en remplissant les champs demander et en soumettant. Les informations saisies sont ensuite vérifiées par le système. Il est impératif que ces informations respectent le format de base attendu pour les entrées. Si les normes ne sont pas respectées, l'utilisateur reçoit un message d'erreur lui indiquant que le format est incorrect, et il est invité à ressaisir les informations. En revanche, si les données saisies sont syntaxiquement valides le système demande à l'utilisateur une confirmation et après confirmation l'expression est mis à jour avec les valeurs saisis et un message de succès est envoyer à l'utilisateur dès lors l'expression est considérée comme traiter.

CU06 : Notifier utilisateur

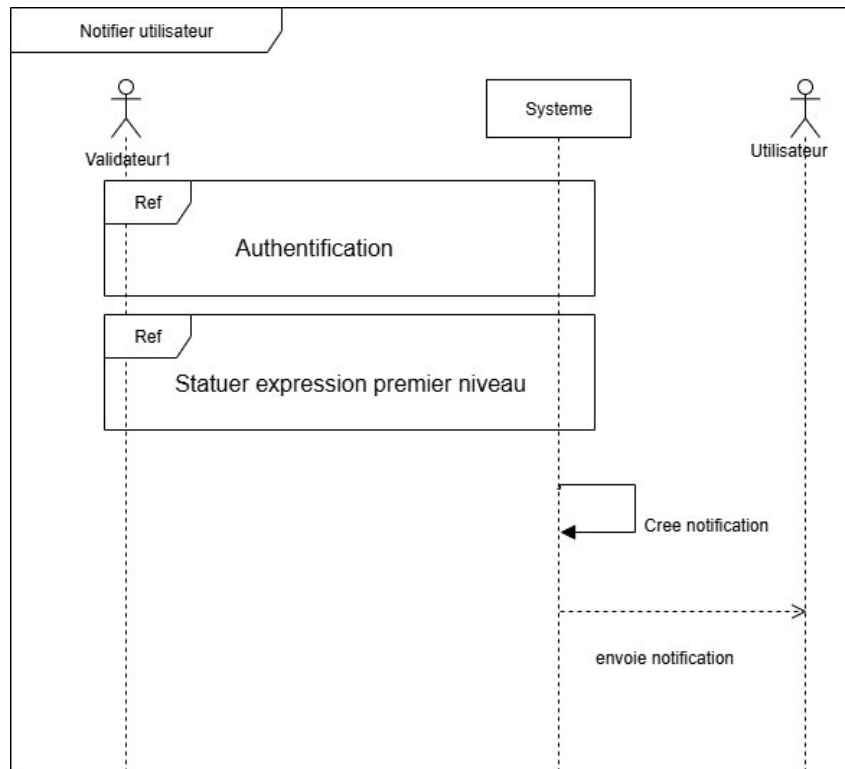


FIGURE 2.9 – Diagramme de séquence du cas d'utilisation « Notifier utilisateur »

La figure 2.9 présente graphiquement les messages échangés au niveau du cas d'utilisation « Notifier utilisateur ». Le valideur, après connexion et après avoir statuer une expression au premier niveau, le système crée une notification et envoie la notification à l'utilisateur. Nous avons illustrer juste un cas de notification pour l'utilisateur mais il en existe plusieurs.

Après avoir défini les différentes fonctionnalités attendues du système à travers les spécifications fonctionnelles, nous abordons à présent les spécifications techniques, qui précisent les choix d'architecture, les technologies utilisées ainsi que les contraintes à respecter pour assurer la robustesse, la performance et la maintenabilité de la solution.

4. Spécification technique

La spécification technique consiste à détailler les choix techniques adoptés afin de répondre aux exigences fonctionnelles et aux besoins définis.

a. Mise à disposition des conditions de travail

L'URCN a mis à notre disposition un serveur pour effectuer nos tests ainsi que la connexion internet pour nos différentes recherches dans le but de parfaire notre travail. Pour le développe-

ment, nous avons utilisé notre ordinateur portable personnel.

b. Architecture de développement

L'architecture logicielle définit la structure et les interactions entre les différents composants d'une application. Elle joue un rôle essentiel dans la stabilité, l'efficacité et la durabilité du logiciel. Une architecture mal conçue peut engendrer des vulnérabilités et des incohérences, notamment lorsqu'elle ne correspond pas au contexte d'utilisation. Parmi les modèles les plus répandus, on retrouve les architectures à deux (2) et trois (3) niveaux. Dans notre cas, le choix s'est porté sur l'architecture trois tiers, car le logiciel Dolibarr utilisé au sein de l'URCN repose déjà sur ce modèle. Étant donné que le module d'expression de besoin doit être intégré à Dolibarr, il est impératif d'adopter la même structure pour garantir la compatibilité et la cohérence du système. Dans la suite, nous présenterons le fonctionnement de l'architecture trois tiers retenue pour ce projet. L'architecture trois tiers est un modèle d'organisation des composants d'un système en trois couches distinctes, chacune ayant un rôle bien défini. Elle est largement utilisée dans les environnements applicatifs modernes, notamment dans les systèmes ERP comme Dolibarr, auquel notre module d'expression de besoin doit s'intégrer. Cette architecture repose sur la séparation des responsabilités en trois niveaux :

- **Couche présentation (1er niveau)** : elle correspond à l'interface utilisateur. C'est à travers cette couche que l'utilisateur interagit avec le système (formulaires, tableaux, boutons, etc.). Elle est généralement développée avec des technologies web comme HTML, CSS, JavaScript ou des frameworks modernes.
- **Couche métier (ou logique applicative (2ieme niveau))** : elle contient les règles de gestion et les traitements liés aux fonctionnalités du système. Elle fait le lien entre les actions de l'utilisateur et les opérations sur les données. C'est ici que se trouvent les validations, les calculs, et les décisions métier.
- **Couche données (3ieme niveau)** : elle gère le stockage, la lecture et la mise à jour des informations dans une base de données. Elle est responsable de la persistance des données et de leur intégrité.

Ces trois couches communiquent entre elles de manière structurée : la couche présentation envoie des requêtes à la couche métier, qui à son tour interagit avec la couche donnée, puis renvoie les résultats vers l'interface utilisateur.

Avantages de l'architecture trois tiers

- **Clarté et organisation** : chaque couche a une responsabilité bien définie, ce qui facilite la compréhension du système.

- **Modularité** : les couches peuvent évoluer indépendamment, ce qui simplifie les mises à jour et les maintenances.
- **Sécurité renforcée** : les accès aux données sont filtrés par la couche métier, limitant les risques d'intrusion ou de corruption.
- **Scalabilité** : il est possible de répartir les couches sur plusieurs serveurs pour améliorer les performances.
- **Réutilisabilité** : les composants métier peuvent être réutilisés dans d'autres interfaces (web, mobile, API).

Limites éventuelles

- **Temps de réponse** : la communication entre les couches peut introduire un léger délai, surtout si elles sont hébergées séparément.
- **Complexité initiale** : la communication entre les couches peut introduire un léger délai, surtout si elles sont hébergées séparément.
- **Maintenance exigeante** : bien que facilitée par la modularité, elle nécessite une bonne coordination entre les équipes techniques.

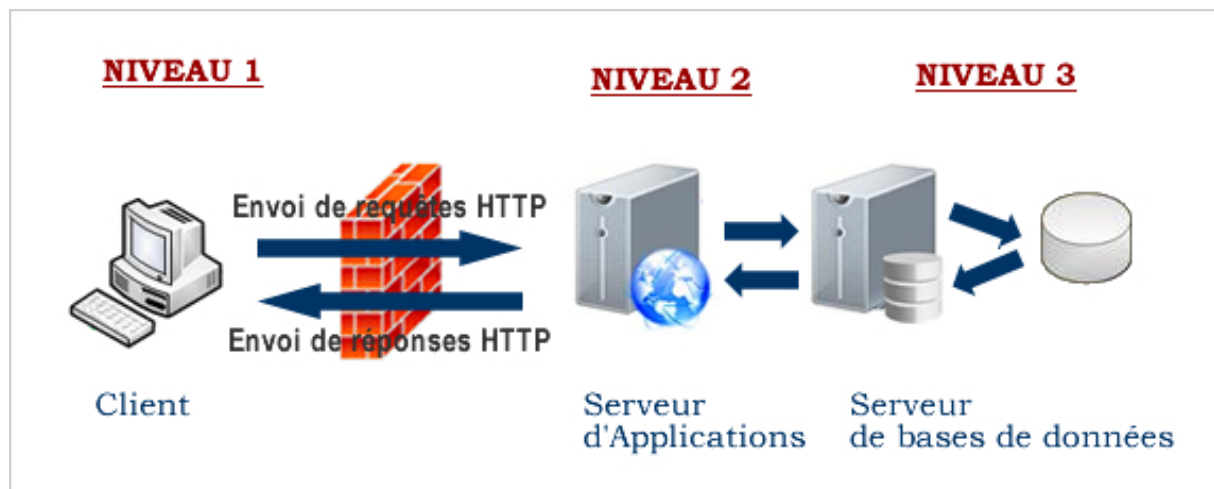


FIGURE 2.10 – Architecture 3 tiers ou à 3 niveaux

Une fois les besoins clairement exprimés et validés, il devient essentiel de traduire ces exigences en une structure technique cohérente. Nous abordons à présent la conception globale du système, qui vise à définir l'architecture, les composants principaux et les interactions nécessaires pour répondre efficacement aux besoins identifiés.

III. Conception globale

Dans cette partie, nous évoquerons le dictionnaire de données, le diagramme des classes ainsi que celui du déploiement.

1. Diagramme des classes et dictionnaire de données

Pour cette partie, nous mettons en évidence notre diagramme des classes ainsi que le dictionnaire de données.

a. Dictionnaire de données

Le dictionnaire de données regroupe l'ensemble des descriptions des attributs associés aux différentes classes utilisées dans le système. Afin de faciliter la lecture et d'éviter toute surcharge d'information, nous avons choisi de présenter ces éléments classe par classe, en nous concentrant exclusivement sur certaines classes métier pour notre module. En effet, le volume global de classes étant relativement important, nous avons retenu celles qui jouent un rôle central dans le fonctionnement du module, à savoir : User, ExpressionBesoin, ExpressionBesoinItem, ExpressionBesoinFingerprint, FournisseurRecommande, Notification.

→Classe : User

Cette classe permet de recueillir les informations d'un utilisateur.

TABLE 2.9 – Description de la classe « User »

Attribut	Description	Type de données
iduser	Identifiant unique de l'utilisateur	Entier
login	Identifiant de connexion	Chaîne de caractères
lastname	Nom de famille de l'utilisateur	Chaîne de caractères
firstname	Prénom de l'utilisateur	Chaîne de caractères
email	Adresse e-mail	Chaîne de caractères
statut	État du compte (actif / non actif)	Booléen
admin	Indique si l'utilisateur est administrateur	Booléen

→**Classe : Notification**

Cette classe permet de recueillir les informations d'une notification.

TABLE 2.10 – Description de la classe « Notification »

Attribut	Description	Type de données
idnotif	Identifiant unique de la notification	Entier
titre	Titre de la notification	Chaîne de caractères
message	Contenu du message	Chaîne de caractères
lue	Indique si la notification a été lue	Booléen
date_envoi	Date d'envoi	Date
url	Lien de redirection lié à la notification	Chaîne de caractères

→**Classe : ExpressionBesoin**

Cette classe permet de recueillir les informations d'une ExpressionBesoin.

TABLE 2.11 – Description de la classe « ExpressionBesoin »

Attribut	Description	Type de données
idexpression	Identifiant unique de l'expression de besoin	Entier
intitule	Titre ou objet de l'expression de besoin	Chaîne de caractères
type_beneficiaire	Type de bénéficiaire	Enumération
imputation	Centre de coût ou budget imputé	Chaîne de caractères
type_urgence	Niveau d'urgence	Enumération
statut	Statut de l'expression (soumis, validé, rejeté. . .)	Enumération
date_creation	Date de création	Date et heure
date_soumission	Date de soumission	Date et heure
date_validation1	Date de validation par le 1er niveau	Date et heure

Suite à la page suivante...

Attribut	Description	Type de données
date_validation2	Date de validation par le 2e niveau	Date et heure
date_transmission	Date de transmission au service achat	Date et heure
chemin_fichier_joint	Chemin du fichier joint	Chaîne de caractères

→ **Classe : ExpressionBesoinFingerprint**

Cette classe permet de stocker les empreintes cryptographiques des documents PDF générés pour assurer leur traçabilité et intégrité.

TABLE 2.12 – Description de la classe « ExpressionBesoinFingerprint »

Attribut	Description	Type de données
idfingerprint	Identifiant unique de l’empreinte	Entier
docuid	Identifiant cryptographique unique du document (DOC-UID)	Chaîne de caractères
hashfile	Empreinte SHA256 du fichier PDF généré	Chaîne de caractères
date_generation	Date et heure de génération du PDF	Date et heure
chemin_fichier	Chemin d’accès au fichier PDF sur le serveur	Chaîne de caractères
hidden_data	Données cachées pour usage futur (version, métadonnées)	Chaîne de caractères
masked_token	Token masqué utilisé pour la vérification via QR code	Chaîne de caractères
contentHash	Signature cryptographique des données métier (articles, validations)	Chaîne de caractères

→ **Classe : ItemExpressionBesoin**

Cette classe permet de recueillir les informations d’un ItemExpressionBesoin.

TABLE 2.13 – Description de la classe « ItemExpressionBesoin »

Attribut	Description	Type de données
iditem	Identifiant unique de l’item	Entier
type_item	Type de l’item (produit, service, etc.)	Enumération
reference	Référence interne/externe de l’item	Chaîne de caractères
designation	Nom ou description courte de l’item	Chaîne de caractères
quantite	Quantité demandée	Nombre décimal
statut	Statut de l’item	Enumération
motif_rejet	Raison du rejet si applicable	Chaîne de caractères
message_achat	Message destiné au service achat	Chaîne de caractères
date_validation1	Date de validation niveau 1	Date et heure
date_validation2	Date de validation niveau 2	Date et heure

→ **Classe : FournisseurRecommande**

Cette classe permet de recueillir les informations d’un FournisseurRecommande.

TABLE 2.14 – Description de la classe « FournisseurRecommande »

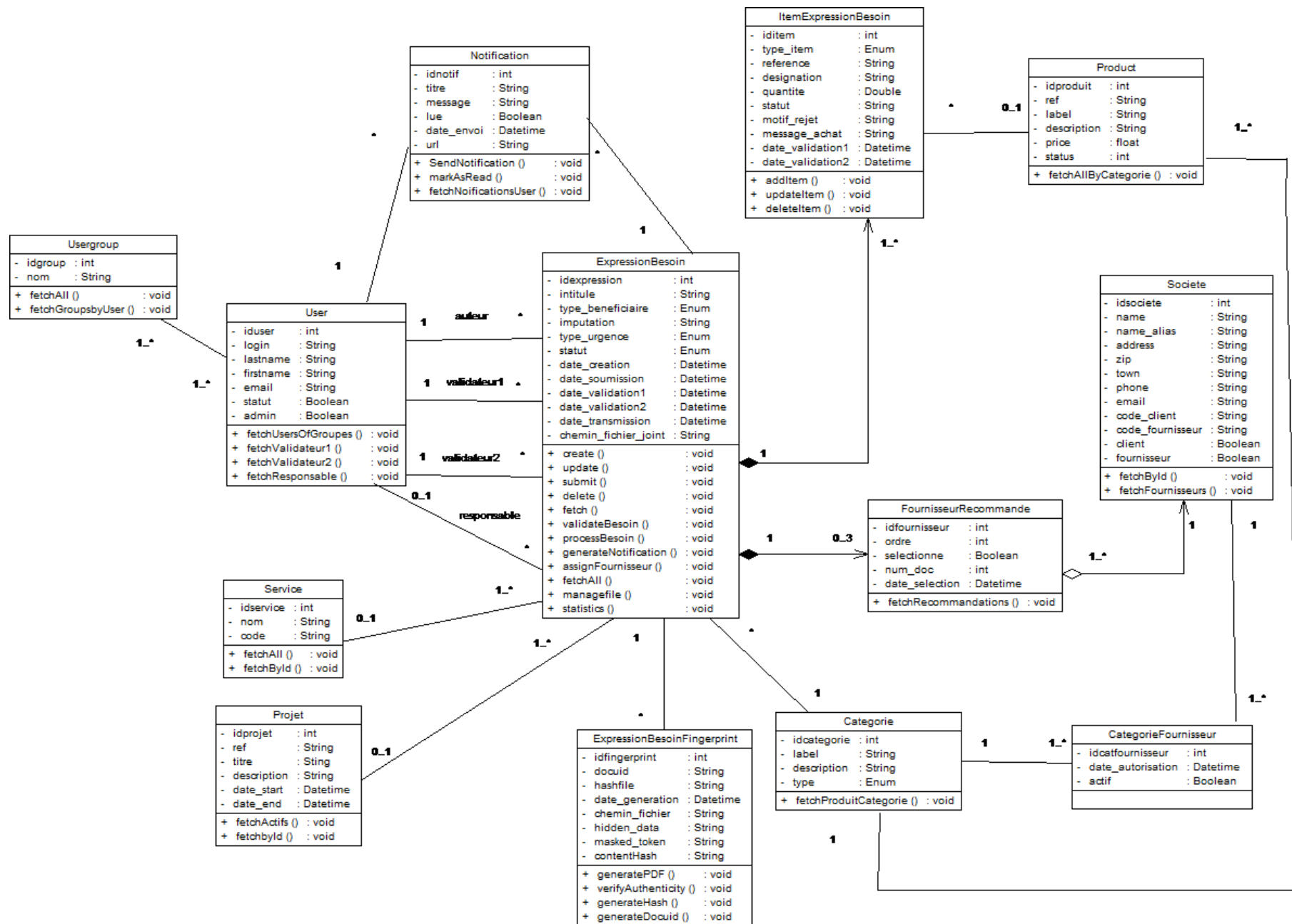
Attribut	Description	Type de données
idfournisseur	Identifiant unique du fournisseur re-commandé	Entier
ordre	Ordre de choix (1, 2, 3)	Entier
selectionne	Indique si ce fournisseur est sélectionné	Booléen
num_doc	Numéro du document de cotation	Entier
date_selection	Date de sélection	Date et heure

Après avoir terminé le dictionnaire de données, nous abordons maintenant le diagramme des classes.

b. Diagramme des classes

Le diagramme des classes exprime la structure statique du système en ce qui concerne les classes et les relations qui existent entre elles. Il constitue la base de la modélisation des données et permet de visualiser à la fois les entités métiers (classes), leurs attributs, ainsi que les liens qui les unissent.

Dans notre application, nous avons identifié treize (13) classes principales, dont : User, Service, Projet, Notification, ExpressionBesoin, ItemExpressionBesoin, Produit, Societe, FournisseurRecommande, Categorie, ExpressionBesoinFingerprint, Usergroup, CategorieFournisseur. Ces classes contiennent les données issues du dictionnaire et interagissent entre elles selon différentes cardinalités.



Règles de gestion associées au diagramme des classes

À partir du diagramme, nous dégagons plusieurs règles de gestion liées aux cardinalités :

- **User (auteur) ↔ ExpressionBesoin** : Un utilisateur peut créer plusieurs expressions de besoin, mais chaque expression est créée par un seul utilisateur.
- **User (validateur1) ↔ ExpressionBesoin** : Un utilisateur peut valider plusieurs expressions, mais chaque expression n'est validée que par un seul validateur de niveau 1.
- **User (validateur2) ↔ ExpressionBesoin** : Un utilisateur peut valider plusieurs expressions, mais chaque expression n'est validée que par un seul validateur de niveau 2.
- **User (responsable) ↔ ExpressionBesoin** : Un responsable achat peut traiter plusieurs expressions, mais une expression peut n'avoir aucun responsable ou au maximum un seul.
- **User ↔ Notification** : Un utilisateur peut recevoir plusieurs notifications, mais chaque notification est liée à un seul utilisateur.
- **ExpressionBesoin ↔ Notification** : Une expression peut être concernée par plusieurs notifications, mais chaque notification est liée à une seule expression.
- **ExpressionBesoin ↔ ItemExpressionBesoin** : Une expression contient au moins un item, et chaque item est lié à une seule expression.
- **ExpressionBesoin ↔ FournisseurRecommande** : Une expression peut contenir jusqu'à trois fournisseurs recommandés, chacun lié à une seule expression.
- **User ↔ Usergroup** : Un utilisateur peut appartenir à plusieurs groupes, et un groupe peut contenir plusieurs utilisateurs.
- **ExpressionBesoin ↔ Service/Projet** : Une expression peut concerner un service ou un projet, chacun pouvant être associé à plusieurs expressions.
- **Produit ↔ Categorie** : Un produit appartient à une seule catégorie, qui peut regrouper plusieurs produits.
- **Societe ↔ Categorie** : Une société peut appartenir à plusieurs catégories, et une catégorie peut regrouper plusieurs sociétés.
- **Societe ↔ FournisseurRecommande** : Une société peut être associée à plusieurs recommandations, mais chaque fournisseur recommandé est lié à une seule société.
- **ExpressionBesoin ↔ ExpressionBesoinFingerprint** : Une expression peut générer aucune ou plusieurs empreintes, chacune liée à une seule expression.
- **ItemExpressionBesoin ↔ Produit** : Un item peut référencer un produit ou aucun, mais chaque produit peut être référencé par plusieurs items.

Ces règles traduisent les contraintes métiers de l'application et garantissent la cohérence entre les différentes entités manipulées par le système.

2. Diagramme de déploiement

Le diagramme de déploiement est un diagramme UML qui montre la configuration physique des différents éléments participant à l'exécution du système, ainsi que les instances de composants qu'ils supportent. Il est constitué de « nœuds » connectés par des liens physiques.

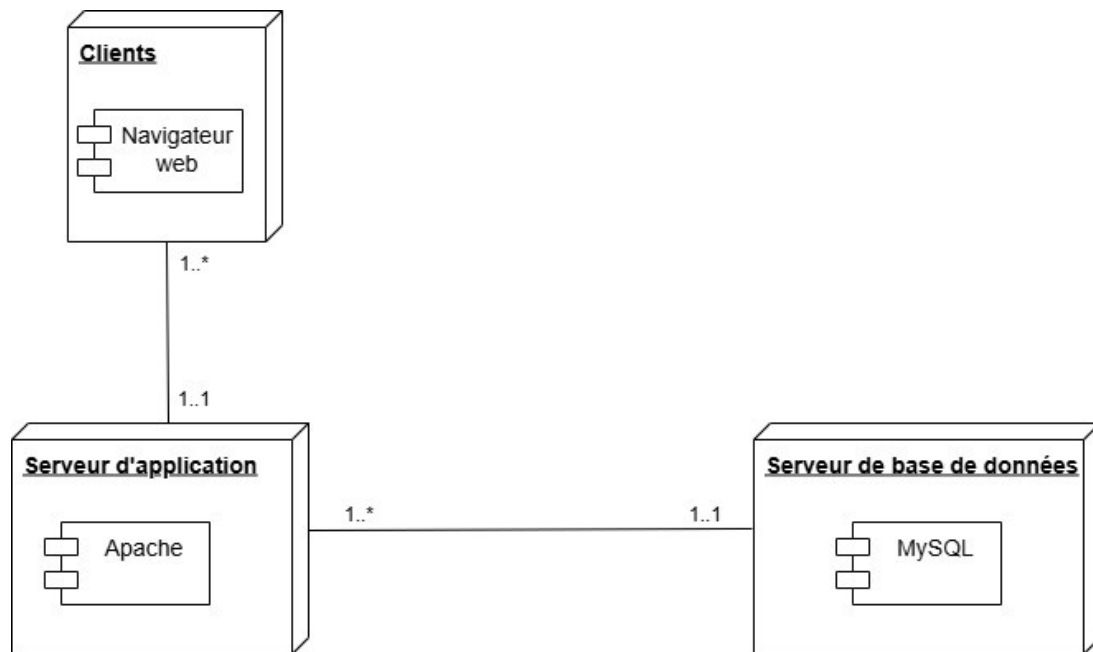


FIGURE 2.12 – Diagramme de déploiement

Notre diagramme de déploiement comprend trois (3) nœuds principaux : le poste client, le serveur d'application (Apache voir Annexe 3) et le serveur de base de données (MySQL voir Annexe 4). Cette structure reflète fidèlement l'architecture de déploiement utilisée par Dolibarr, le logiciel déjà en place au sein de l'URCN.

Étant donné que le module d'expression de besoin que nous développons doit être intégré à Dolibarr, il est impératif de respecter cette même architecture afin d'assurer une compatibilité optimale et une intégration fluide.

Chaque nœud joue un rôle bien défini dans le fonctionnement global du système, et leur répartition permet une meilleure organisation, une maintenance facilitée et une évolutivité maîtrisée.

Une fois la conception globale du système établie, il est temps de passer à la phase de réalisation, qui consiste à concrétiser les éléments définis précédemment. Cette étape inclut la mise en œuvre des composants, leur codage, les tests unitaires, ainsi que l'intégration progressive des fonctionnalités dans l'environnement cible.

IV. Réalisation

Plusieurs éléments importants entrent en compte dans la réalisation d'un logiciel. Ces éléments peuvent être le choix des outils de réalisation, les maquettes, les politiques de sécurité et le coût de réalisation de la solution. Nous les mettrons en lumière dans cette partie de l'étude.

1. Présentation des outils de réalisation

Les outils de réalisation sont des éléments qui aident un développeur dans le déroulement d'une activité de développement. Parmi ces éléments, nous étudierons notamment les langages de programmation utilisés et les outils de conception.

a. Langages de programmation

Un algorithme est une suite d'opérations logiques permettant d'atteindre un objectif précis. En informatique, ces algorithmes sont traduits et exécutés à l'aide de langages de programmation, qui servent à formaliser les instructions destinées à la machine.

Dans le cadre du développement de notre application, plusieurs langages ont été utilisés :

- **PHP** : langage libre et gratuit, principalement utilisé côté serveur. Il permet de générer dynamiquement du contenu web tel que du HTML, du CSS, du XHTML, ainsi que des fichiers de données (images au format PNG, JPG, etc.) ou des documents PDF.
- **JavaScript** : langage de script incontournable dans le développement web, utilisé pour rendre les pages interactives. Il fonctionne en complément de HTML et CSS, formant avec eux le trio fondamental du web moderne.
- **HTML5** : langage de balisage utilisé pour structurer et afficher le contenu des pages web.
- **CSS3** : langage de style qui permet de définir l'apparence visuelle des pages web, notamment la mise en forme, les couleurs, les polices et les dispositions.

b. Outils de conception

Les outils de conception regroupent l'ensemble des logiciels et plateformes utilisés tout au long du projet, depuis la phase de modélisation jusqu'à la réalisation technique. Dans notre cas, les outils suivants ont été mobilisés :

- **Draw.io** : utilisé pour la création des différents diagrammes UML (cas d'utilisation, séquences, classes, activités, etc.). Cet outil en ligne, open source et multiplateforme,

nous a permis de modéliser de manière claire et structurée les aspects fonctionnels et techniques du système.

- **PowerAMC** : utilisé pour la modélisation du diagramme des classes. Cet outil professionnel, développé par SAP, est largement reconnu dans le domaine de l'ingénierie logicielle pour sa puissance et sa précision. Il permet de concevoir des modèles conceptuels, logiques et physiques de données, tout en respectant les standards UML.
- **Git 2.39.0** : logiciel de gestion de versions décentralisé, utilisé pour suivre l'évolution du code source, archiver les modifications et faciliter la collaboration. Il nous a permis de maintenir une trace fiable de l'ensemble des développements réalisés.
- **Microsoft Office Word et PowerPoint 2016** : utilisés pour la rédaction de la documentation du projet et la préparation des supports de présentation. Word a servi à formaliser les spécifications et les rapports, tandis que PowerPoint a été utilisé pour illustrer les étapes clés du projet.
- **LaTeX** : utilisé pour la rédaction du rapport final et la génération du document PDF. Ce langage de composition typographique permet de produire des documents professionnels, bien structurés et adaptés aux exigences académiques. Grâce à LaTeX, nous avons pu intégrer des tableaux complexes, des diagrammes, des références croisées et une mise en page cohérente tout au long du document.
- **PhpMyAdmin version 4.8.3 (via XAMPP voir Annexe 5)** : utilisé pour l'administration de la base de données MySQL. Cet outil web nous a permis de gérer les tables, exécuter des requêtes SQL, importer/exporter des données et assurer le bon fonctionnement du stockage des informations.
- **Visual Studio Code** : éditeur de code léger, open source et extensible, développé par Microsoft. Il a été utilisé pour écrire, organiser et tester le code du module. Son interface intuitive et ses nombreuses extensions ont facilité le développement des différentes fonctionnalités.
- **Postman version 10.0.1** : utilisé pour tester les requêtes envoyées à notre API backend. Cet outil nous a permis de simuler les appels, vérifier les réponses du serveur et valider le bon fonctionnement des échanges entre les composants.

Après avoir présenté les différents outils utilisés tout au long du projet, il est désormais essentiel de s'intéresser à la structure interne de l'application. Celle-ci repose sur le modèle MVC, qui permet d'organiser le code de manière claire, modulaire et évolutive.

2. Présentation de l'architecture MVC de notre application

Le module d'expression de besoin que nous avons développé s'intègre directement au logiciel Dolibarr, qui repose sur une architecture Modèle–Vue–Contrôleur (MVC). Cette architecture

est une exigence fondamentale pour tout module Dolibarr, car elle garantit la cohérence, la maintenabilité et l'interopérabilité des composants au sein de la plateforme.

L'approche MVC permet de structurer l'application en trois parties distinctes : le **Modèle**, la **Vue** et le **Contrôleur**. Cette séparation facilite la gestion du code, en isolant la logique métier de la présentation, tout en assurant une meilleure évolutivité du système.

- **Modèle** : représente les données de l'application ainsi que les règles métier associées. Il permet de gérer les expressions de besoin, d'interagir avec la base de données MySQL, et de garantir l'intégrité des informations.
- **Vue** : correspond à l'interface utilisateur. Elle affiche les informations issues du modèle et transmet les actions de l'utilisateur au contrôleur. Elle est construite en HTML, JavaScript et PHP, sans contenir de logique métier.
- **Contrôleur** : assure la liaison entre le modèle et la vue. Il reçoit les actions de l'utilisateur, interagit avec le modèle pour traiter les données, puis transmet les résultats à la vue pour affichage.

Cette architecture, que propose Dolibarr pour les modules, nous a permis de développer un module robuste, structuré et parfaitement intégré à l'environnement existant. Elle facilite également la maintenance future et l'ajout de nouvelles fonctionnalités sans perturber l'ensemble du système.

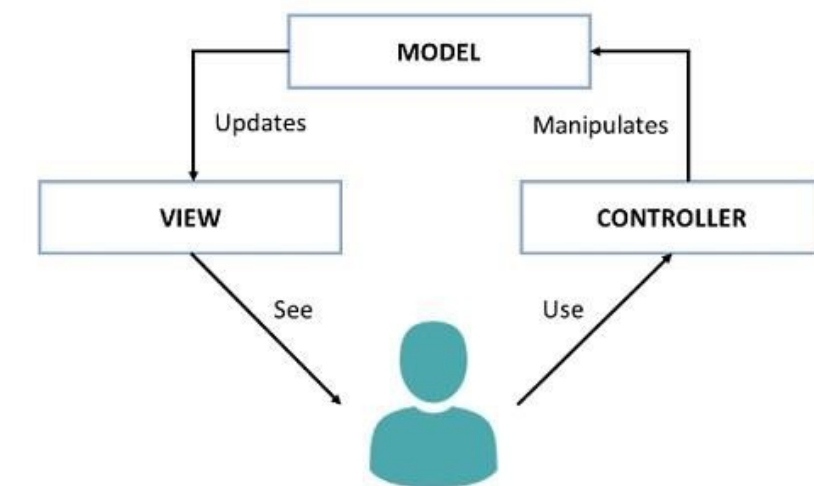


FIGURE 2.13 – Structure de l'architecture Modèle–Vue–Contrôleur

Après avoir présenté l'architecture et les fondements techniques de notre application, il est désormais pertinent de visualiser concrètement l'interface utilisateur telle qu'elle sera perçue par les acteurs du système. Dans la section suivante, nous présentons quelques maquettes IHM de notre module, illustrant les principales interactions et écrans clés du parcours utilisateur.

3. Quelques maquettes IHM de notre application

Dans cette section, nous présentons les maquettes de l'interface utilisateur (IHM) de notre module, afin d'illustrer visuellement les principales interactions entre l'utilisateur et le système. Ces maquettes permettent de mieux comprendre l'organisation des écrans, la navigation proposée, ainsi que l'ergonomie générale de l'application.

L'IHM repose principalement sur quatre écrans clés, correspondant aux étapes essentielles du processus :

- **La page de connexion** : permet à l'utilisateur de s'authentifier pour accéder au système.
- **La page de création d'expression** : dédiée à la saisie et à la soumission d'une nouvelle expression de besoin.
- **La page de validation** : utilisée par les acteurs habilités pour statuer sur les expressions soumises.
- **La page de traitement** : permet de finaliser et de suivre l'évolution des expressions validées.

À noter

Certains écrans, en raison de leur richesse fonctionnelle et de la densité des informations qu'ils affichent, occupent une surface importante. Ainsi, lors de la capture d'écran, il peut arriver que certaines parties ne soient pas entièrement visibles sur une seule image. Cette contrainte est liée à la complexité et à l'étendue des interfaces concernées.

→ Connexion

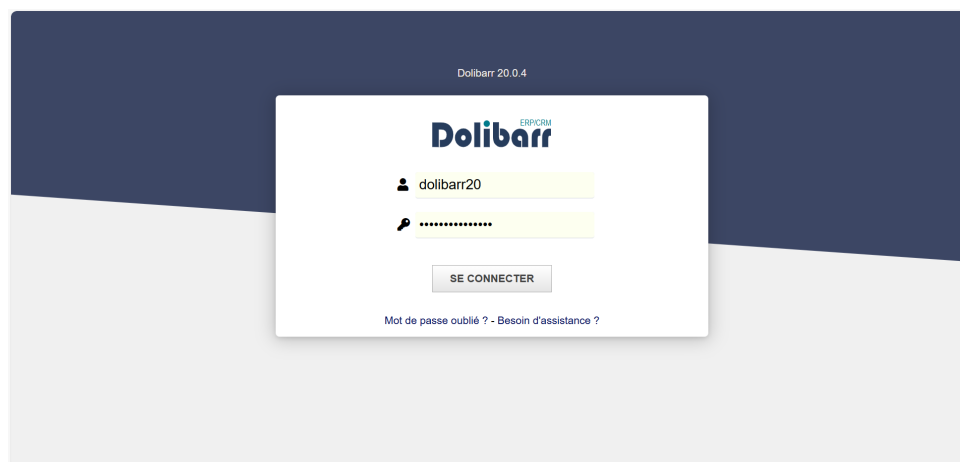


FIGURE 2.14 – Écran de connexion

La figure 2.14 représente la partie connexion de Dolibarr. Tous les utilisateurs ayant un

compte peuvent se connecter à partir de ce formulaire et accéder au module, à condition que celui-ci ait été activé au préalable.

→ Création d'expression

Créer une expression

Libellé

BeneficiaryType: Projet

Projet

Urgence: Normal

Validators: Paramètres manquants

Imputation: ImputationExample

Tag/catégorie: Sélectionner une catégorie

Items 1

#	Type	Désignation	Référence	Quantité	Action
1	Existe	SelectItem	Référence	1	

AJOUTER UN ITEM

FIGURE 2.15 – Écran de création d'expression

La figure 2.15 illustre l'écran dédié à la création d'une expression de besoin. Une fois connectés, les utilisateurs peuvent accéder au module et sélectionner le menu « Nouvelle expression ». Un formulaire de saisie s'affiche alors, qu'ils peuvent remplir avant de choisir entre deux options : soumettre l'expression pour validation ou l'enregistrer en tant que brouillon pour un traitement ultérieur.

→ Validation d'expression

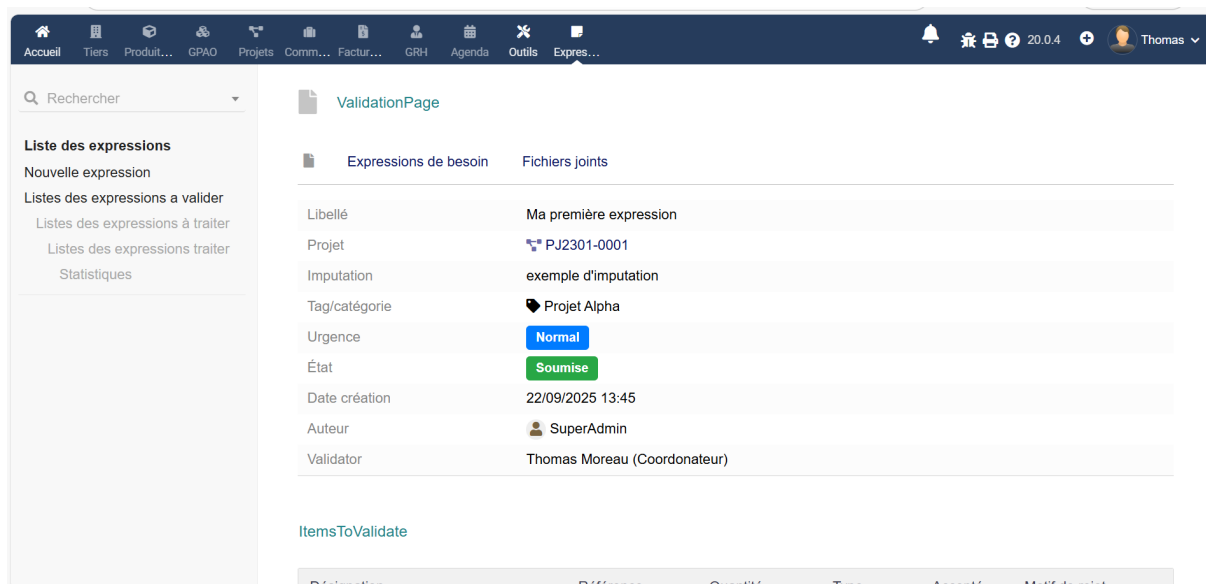


FIGURE 2.16 – Écrans de validation d’expression

La figure 2.16 illustre l’écran dédié à la validation d’une expression de besoin. Une fois connecté, si l’utilisateur est désigné comme validateur, il peut accéder à la liste des expressions à valider. Il lui suffit alors de sélectionner l’expression concernée pour accéder à la page de validation. Sur cette page, le validateur peut consulter les détails, suivre les instructions affichées, puis cliquer sur le bouton « Signer » afin de confirmer sa validation.

→ Traitement d’expression

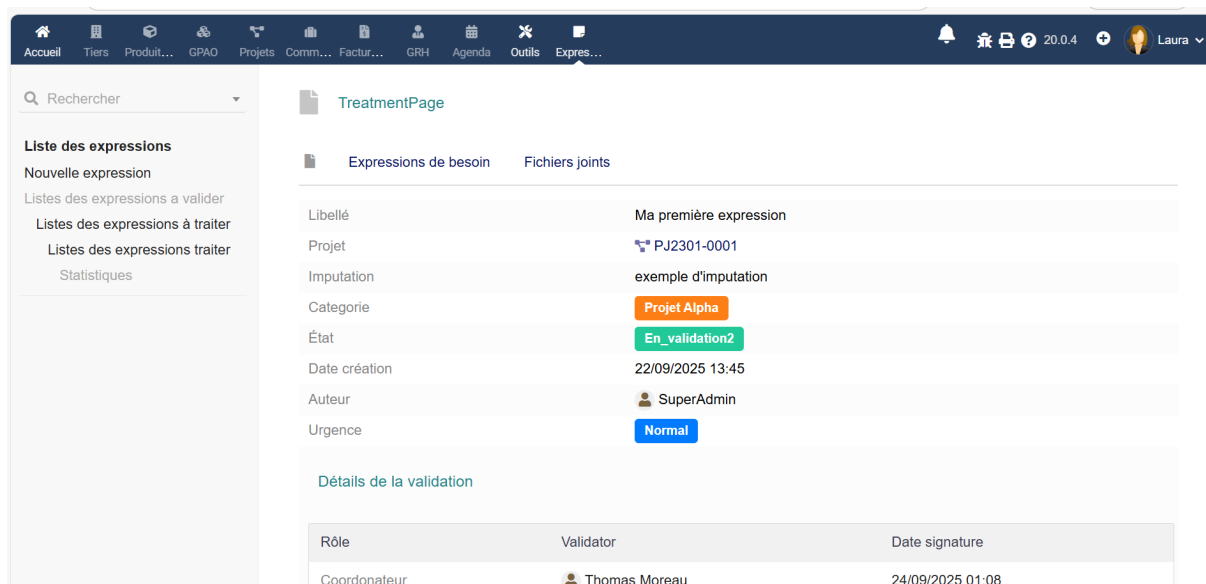


FIGURE 2.17 – Écran de traitement d’expression

La figure 2.17 illustre l’écran dédié au traitement des expressions de besoin par les responsables. Une fois connecté, si l’utilisateur est identifié comme responsable achat, il peut accéder à

la liste des expressions à traiter. Il lui suffit de sélectionner l'expression concernée pour accéder à la page de traitement, où il peut consulter les détails, effectuer les actions nécessaires selon les consignes affichées, puis finaliser le processus en cliquant sur le bouton « Signer » pour confirmer le traitement.

Après avoir conçu et visualisé les interfaces utilisateur à travers les maquettes IHM, il est essentiel de s'assurer que le système fonctionne conformément aux attentes. Nous abordons à présent la phase de tests, qui vise à vérifier la fiabilité, la stabilité et la conformité des différentes fonctionnalités développées. Cette étape permet de détecter d'éventuelles anomalies et de garantir une expérience utilisateur optimale avant le déploiement final.

4. Tests

Les tests occupent une place essentielle dans le processus de développement logiciel. Ils permettent de vérifier que les différentes fonctionnalités implémentées répondent aux besoins exprimés, tout en garantissant la qualité, la stabilité et la fiabilité de l'application.

Qu'il s'agisse de tests unitaires, fonctionnels ou de robustesse, chacun contribue à s'assurer que les composants du système interagissent correctement et produisent les résultats attendus. Conscients de leur importance, nous avons mis en œuvre une série de tests ciblés sur les fonctionnalités critiques de notre module, afin de valider son bon fonctionnement avant toute mise en production.

Cette démarche vise à renforcer la confiance dans la solution développée et à anticiper d'éventuelles anomalies ou dysfonctionnements.

a. Outils de test

Étant donné que notre module est développé en PHP pur, sans framework externe comme Laravel, nous avons opté pour des outils simples, efficaces et compatibles avec l'environnement de Dolibarr :

- **PHPUnit** : framework de test officiel pour PHP, utilisé pour les tests unitaires. Il permet de vérifier le bon fonctionnement des classes et méthodes du module.
- **ModuleBuilder (Dolibarr)** : générateur automatique de fichiers de test unitaire pour chaque classe créée, compatibles avec PHPUnit.

Ces outils nous ont permis de réaliser des tests ciblés et pertinents, adaptés à notre environnement de développement, tout en respectant les standards de qualité attendus pour une intégration fluide dans Dolibarr.

b. Mise en place des tests

Deux types de tests ont été mis en œuvre :

- Tests unitaires automatisés avec PHPUnit et ModuleBuilder
- Tests fonctionnels réalisés manuellement
- Une matrice de couverture reliant chaque test aux exigences fonctionnelles

→ Tests unitaires

Les tests unitaires ont été réalisés avec PHPUnit, installé via Composer avec la commande : `composer require -dev phpunit/phpunit`. Grâce à ModuleBuilder, les fichiers de test ont été générés automatiquement dans le répertoire `test/phpunit`, puis personnalisés pour couvrir les scénarios critiques du module.

TABLE 2.15 – Jeu de test unitaire prévisionnel

ID du Test	Description	Méthode testée	Entrée	Sortie attendue
TU001	Création d'une expression	<code>create()</code>	Données valides	<code>result > 0</code>
TU002	Soumission incomplète	<code>submit()</code>	Données incomplètes	Message : Soumission refusée
TU003	Validation par un validateur	<code>validateBesoin()</code>	ID + rôle	<code>true</code>
TU004	Mise à jour du statut	<code>update()</code>	Expression + nouveau statut	<code>result > 0</code>
TU005	Traitement de l'expression	<code>processBesoin()</code>	ID + données de traitement	<code>true</code>
TU006	Attribution d'un fournisseur	<code>assignFournisseur()</code>	ID + société	<code>true</code>
TU007	Déclenchement d'une notification	<code>generateNotification()</code>	ID + rôle	Notification générée

Les tests ont été exécutés avec succès. Voici une capture du terminal illustrant les résultats obtenus, dans la figure 2.18.

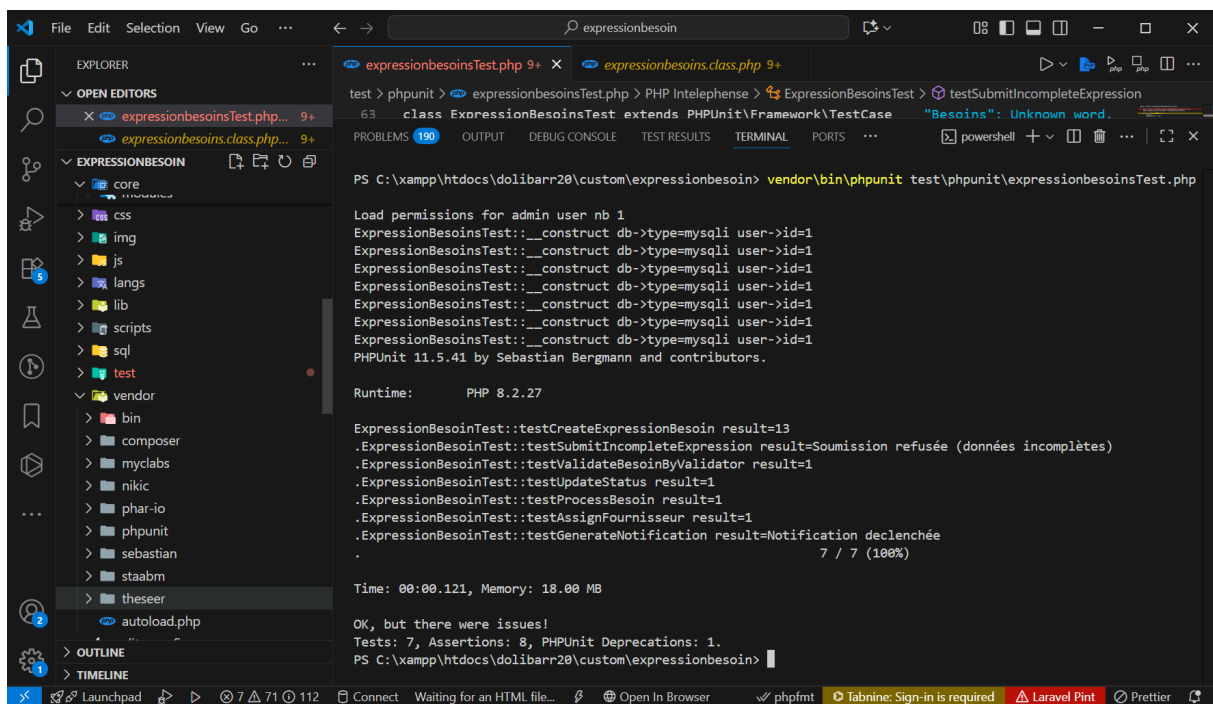


FIGURE 2.18 – Résultat obtenu

Les sept tests unitaires ont été exécutés avec succès. Les méthodes `create()`, `submit()`, `update()`, `validateBesoin()`, `processBesoin()`, `assignFournisseur()`, `generateNotification()` ont toutes retourné les résultats attendus.

TABLE 2.16 – Jeu de test unitaire final

ID du Test	Description	Résultat attendu	Résultat obtenu	Constat
TU001	Création d'une ex-pression	result > 0	result = 13	Réussi
TU002	Soumission incom-plète	Message : Soumis-sion refusée	Soumission refusée	Réussi
TU003	Validation par un va-lidateur	true	1	Réussi
TU004	Mise à jour du statut	result > 0	result = 1	Réussi
TU005	Traitement de l'ex-pression	true	1	Réussi
TU006	Attribution d'un fournisseur	true	1	Réussi
TU007	Déclenchement d'une notification	Notification générée	Notification déclen-chée	Réussi

→ **Tests fonctionnels**

Les tests fonctionnels ont été réalisés manuellement, en simulant les actions des utilisateurs dans l'interface du module.

TABLE 2.17 – Jeu de test fonctionnel

ID du Test	Description	Actions	Résultat attendu	Résultat obtenu	Constat
TF001	Création avec données complètes	Connexion, accès au module, saisie complète, soumission ou brouillon, confirmation	Message de succès et redirection	Code 302	Réussi
TF002	Création avec données incomplètes	Connexion, accès au module, saisie partielle, soumission, confirmation	Message d'erreur et retour sur la page	Code 422	Réussi
TF003	Validation d'une expression	Connexion en tant que validateur, sélection, avis, signature, confirmation	Message de succès et redirection	Code 302	Réussi
TF004	Traitement complet	Connexion en tant que responsable, sélection, remplissage complet, signature, confirmation	Message de succès et redirection	Code 302	Réussi
TF005	Traitement incomplet	Connexion en tant que responsable, sélection, remplissage partiel, signature, confirmation	Message d'erreur et retour sur la page	Code 422	Réussi

Les résultats ont été vérifiés à deux niveaux : d'abord via les outils de développement du navigateur, où nous avons observé les codes HTTP comme 302 (redirection après soumission réussie) ou 422 (erreur de validation); ensuite via les logs serveur, qui confirment que les requêtes ont été bien traitées.

Après avoir fini les tests fonctionnels nous passons à la matrice de couverture

→ **Matrice de couverture** La matrice de couverture permet de relier chaque test aux exigences fonctionnelles du module. Elle garantit que chaque exigence est vérifiée par au moins un test. La voici dans le tableau 2.18.

NB

EX1, EX2 à EX6 sont les exigences que nous avons eues à définir un peu plus haut au niveau de l'étude de l'existant.

TABLE 2.18 – Matrice de couverture des tests par exigence fonctionnelle

ID du Test	EX1	EX2	EX3	EX4	EX5	EX6
TU001	X					
TU002				X		
TU003			X	X		
TU004				X		
TU005				X		
TU006				X		
TU007						X
TF001	X					
TF002		X				
TF003			X	X	X	
TF004				X	X	
TF005		X				

Chaque exigence (EX1 à EX6) est couverte par au moins un test, ce qui garantit une bonne couverture fonctionnelle. Au-delà des tests déjà réalisés, notamment les tests unitaires et fonctionnels effectués dans notre environnement de développement local, il est important de souligner que l'intégration du module n'a pas encore été testée en environnement réel. Des tests complémentaires tels que les tests d'intégration sur le serveur de production de l'URCN, ainsi que des tests de performance, de charge et de sécurité seront menés une fois le module déployé dans son contexte opérationnel. Ces tests permettront de valider le bon fonctionnement du module dans des conditions réelles d'utilisation, avec les véritables données, rôles et infrastructures de l'URCN. Par ailleurs, au-delà de la validation fonctionnelle, il est essentiel de garantir que le module respecte les principes fondamentaux de sécurité. La protection des données, la gestion

des accès et la conformité aux bonnes pratiques sont des enjeux majeurs dans tout environnement professionnel, en particulier dans le domaine de la recherche clinique. Nous abordons donc dans la section suivante la politique de sécurité appliquée au module, en détaillant les mécanismes mis en place pour assurer la confidentialité, l'intégrité et la traçabilité des informations manipulées.

5. Politique de sécurité

La politique de sécurité du module d'expression de besoin vise à garantir la confidentialité, l'intégrité et la traçabilité des données tout au long du cycle de vie de l'application. Elle repose sur plusieurs piliers : la sensibilisation des utilisateurs, la sécurisation du développement, la protection des échanges, la traçabilité des documents générés, la journalisation des actions, et la sauvegarde des données.

a. Sensibilisation des utilisateurs

Le facteur humain étant souvent à l'origine des failles de sécurité, il est essentiel de sensibiliser les utilisateurs aux bonnes pratiques. À travers des recommandations simples, le module encourage les comportements responsables :

- Ne jamais partager ses identifiants avec des tiers.
- Ne pas laisser une session ouverte sans surveillance.
- Mettre à jour régulièrement les antivirus et logiciels de sécurité.
- Éviter les connexions à des réseaux non sécurisés ou à des sites sans certificat SSL.

b. Sécurité durant le développement

Le module repose sur l'architecture de Dolibarr, qui intègre une gestion fine des droits d'accès par profil utilisateur. Chaque action (création, validation, traitement) est conditionnée par le rôle de l'utilisateur. Des contrôles de saisie sont intégrés dans les formulaires pour empêcher l'enregistrement de données invalides.

c. Traçabilité et Sécurisation des Documents PDF

Afin d'assurer la fiabilité des documents générés, un dispositif de sécurité a été mis en œuvre. Il repose sur des mécanismes cryptographiques garantissant l'intégrité des données métier et la traçabilité des validations.

— Principes de Sécurisation

- **Identifiant unique sécurisé** : Chaque document est associé à un identifiant cryptographique généré de manière déterministe.
- **Token opaque** : Une référence masquée permet d'assurer la confidentialité des identifiants métiers lors de la vérification.
- **Signature des données métier** : Une empreinte cryptographique stable est calculée à partir du contenu métier critique (articles, demandeur, date, urgence).
- **Vérification par référence** : La signature intégrée au QR code est comparée à celle recalculée depuis la base de données officielle.
- **Empreinte persistante** : Chaque génération de PDF est enregistrée avec ses métadonnées de signature dans une table dédiée.
- **Protection cryptographique** : Une clé maître dérivée de l'environnement applicatif sécurise la génération des signatures.
- **Détection d'altération** : Toute modification des données métier en base est détectée par divergence des signatures.

— Protocole de Vérification

Le système applique une procédure en trois étapes :

1. **Référencement** : Identification du document officiel via le token sécurisé
2. **Recalcul** : Régénération de la signature à partir des données métier en base de données
3. **Comparaison** : Vérification cryptographique de la cohérence entre la signature embarquée et celle recalculée

— Garanties de Sécurité

Ce dispositif assure les propriétés suivantes :

- **Intégrité des données** : Toute modification des informations métier (articles, quantités, etc.) est immédiatement détectée
- **Traçabilité des validations** : Les signatures et dates de validation sont sécurisées contre toute altération
- **Référencement sécurisé** : Le système garantit que la vérification pointe vers le document officiel enregistré
- **Non-répudiation technique** : La signature cryptographique empêche de contester la version des données au moment de la génération

— Portée et Limites

Le système assure principalement l'intégrité des données métier enregistrées dans l'application. La vérification confirme que les informations affichées correspondent à la version officielle en base de données, établissant ainsi une chaîne de confiance entre le document physique et le système informatique.

d. Journalisation des actions

Dolibarr propose un module « Archives/Logs Inaltérables » permettant de conserver un historique sécurisé et non modifiable des actions. Ce module est activé dans l'environnement réel de l'URCN.

e. Sécurisation des échanges (SSL)

Tous les échanges sont protégés par HTTPS via certificat SSL. Le serveur et les API forcent la redirection vers HTTPS pour empêcher toute communication non sécurisée.

f. Politique de sauvegarde

Une politique de sauvegarde structurée est appliquée :

- Sauvegardes journalières : 7 jours
- Sauvegardes hebdomadaires : 1 mois
- Sauvegardes mensuelles : 6 mois
- Sauvegardes semestrielles : 1 an
- Sauvegardes annuelles : archivées de manière permanente

6. Coût de réalisation

Après avoir défini les mesures de sécurité, il est essentiel d'évaluer le coût de développement du projet. Cette estimation permet de mesurer l'effort requis, la durée de réalisation, et les ressources financières mobilisées. Elle constitue une base de référence pour anticiper les besoins budgétaires et éviter les dépassements.

a. Étude comparative des méthodes d'estimation

La conception d'un logiciel inclut une phase cruciale : l'estimation du coût de réalisation. Cette étape permet d'anticiper les ressources humaines, financières et matérielles nécessaires pour mener à bien le projet. Il existe de nombreuses méthodes d'estimation, mais nous avons analysé les plus populaires pour identifier celles adaptées à notre contexte.

TABLE 2.19 – Comparaison des méthodes d'estimation de coût logiciel

Méthode	Principes	Avantages	Inconvénients
Estimation par analogie	Comparaison avec des projets similaires	Bonne précision si les données comparables sont disponibles	Inapplicable si aucun projet similaire n'existe
Jugement d'experts	Estimation basée sur l'expérience	Rapide et flexible, utile en phase préliminaire	Subjectif, dépend fortement du niveau d'expertise
COCOMO	Calcul basé sur lignes de code et type de logiciel	Méthode algorithmique, populaire, facile à appliquer	Ignore les contraintes matérielles et humaines
Delphi	Consensus entre experts via itérations anonymes	Réduit les biais individuels, améliore la fiabilité	Long à mettre en œuvre, nécessite plusieurs experts
Machine Learning	Modèles prédictifs sur données historiques	Haute précision si les données sont fiables	Complexité technique, nécessite données volumineuses

À l'issue de cette analyse comparative, plusieurs méthodes ont été écartées pour notre contexte académique :

- Le Jugement d'experts et la méthode Delphi nécessitent une expertise approfondie difficile à obtenir dans notre cas
- Les approches par Machine Learning requièrent des données historiques volumineuses et complexes à obtenir

Deux méthodes ont particulièrement retenu notre attention : COCOMO et l'Estimation par analogie. Nous avons calculé les résultats selon ces deux approches (voir Annexe 2 pour les détails des calculs).

b. Résultats comparés et choix de la méthode

Les calculs détaillés en annexe nous donnent les estimations suivantes :

- **Méthode COCOMO** : 6,63 mois de développement
- **Méthode par analogie basée sur la réalisation effective** : 3,8 mois de développement

Après analyse comparative, la méthode par analogie basée sur la réalisation effective apparaît comme la plus fiable et adaptée à notre contexte. En effet, cette approche s'appuie sur le développement concret du module durant la période de stage de 3 mois, avec un état d'avancement de 90% des fonctionnalités. Nous retenons donc l'estimation de 3,8 mois pour la suite de notre analyse.

c. Coût global du projet

En appliquant l'estimation basée sur la réalisation effective et en considérant un salaire mensuel moyen de 250 000 FCFA pour un développeur PHP, nous obtenons :

- Coût de développement = 3,8 mois × 250 000 FCFA = 950 000 FCFA

En complément du coût de développement, nous avons pris en compte les dépenses liées au matériel et aux outils utilisés. La plupart des logiciels employés sont open source ou gratuits, ce qui limite les coûts annexes.

TABLE 2.20 – Coût total du projet

Désignation	Quantité	Coût unitaire	Coût total
Ordinateur de développement	1	450 000 FCFA	450 000 FCFA
Licences logicielles (Project 2016)	1	113 000 FCFA	113 000 FCFA
Outils gratuits (Draw.io, VS Code, MySQL)	-	-	-
Développement (réalisation effective)	3,8 mois	250 000 FCFA/mois	950 000 FCFA
Coût total	–	–	1 513 000 FCFA

Le coût total de réalisation du module d'expression de besoin est estimé à 1 513 000 FCFA, en tenant compte du développement et du matériel nécessaire. Cette estimation, basée sur la réalisation effective du module, reste raisonnable au regard des fonctionnalités livrées, de la sécurité intégrée, et de la traçabilité assurée par le système.

7. Planning réel

Au début de ce chapitre, nous avons exposé notre planification anticipée sur laquelle nous sommes appuyés pour l'exécution des différentes tâches. Cependant, certaines tâches ont été achevées plus tôt que prévu, tandis que d'autres ont pris plus de temps que prévu, en raison

de circonstances indépendantes de notre volonté. Par conséquent, le planning effectif se présente comme suit :

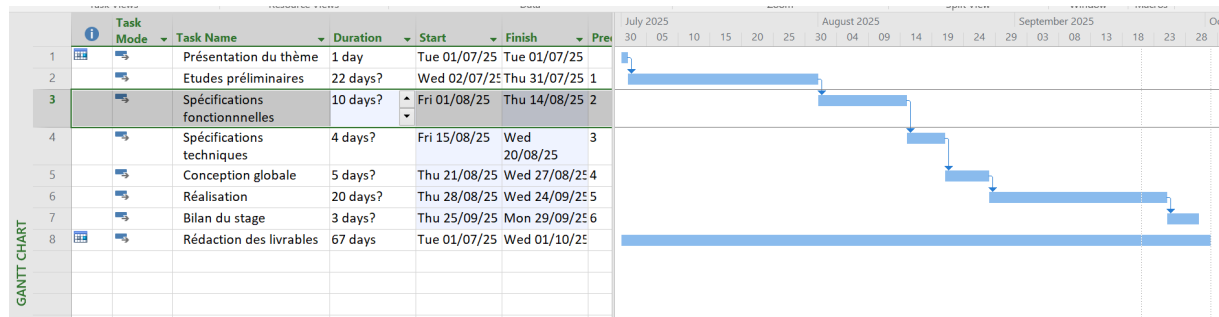


FIGURE 2.19 – Planning réel

Dans ce chapitre, nous avons examiné de manière détaillée toutes les procédures et ressources nécessaires à la création de notre module. Pour conclure cette étape et passer à la dernière phase de notre travail, le troisième et dernier chapitre se penchera sur l'évaluation du déroulement de notre stage.

Chapitre 3

BILAN DU STAGE

Dans ce chapitre final, nous présentons le bilan de notre stage. Nous mettons en évidence, d'une part, le déroulement du stage et les activités menées, et d'autre part, les connaissances acquises, nos observations ainsi que les perspectives qui en découlent.

I. Présentation du déroulement du stage et des activités réalisées

1. Activités menées

Notre stage s'est déroulé à l'Unité de Recherche Clinique de Nanoro (URCN), du 1^{er} juillet au 30 septembre 2025.

Au cours de cette période, nous avons principalement travaillé sur l'étude, la compréhension et l'extension de l'ERP Dolibarr, en développant un module spécifique adapté aux besoins de l'URCN.

Les principales activités réalisées ont été :

- Étude de l'existant et exploration d'une version bêta de la base de données de Dolibarr, ce qui nous a permis de constater l'envergure des données et de nous familiariser avec le nombre considérable de tables.
- Découverte et apprentissage de la structuration interne de Dolibarr (organisation des modules, format des fichiers, logique de développement).
- Développement d'un module personnalisé pour répondre aux besoins identifiés.
- Documentation du processus et mise en évidence des limites techniques rencontrées.

2. Connaissances acquises

Ce stage a été une véritable première immersion dans le monde des ERP, et plus particulièrement dans Dolibarr. Nous avons pu :

- Découvrir le fonctionnement interne d'un ERP ancien mais encore pertinent, et en com-

prendre la philosophie.

- Approfondir nos compétences en PHP et en manipulation de bases de données volumineuses.
- Développer une capacité d'analyse technique pour identifier les besoins et proposer des solutions adaptées.
- Renforcer notre rigueur dans le travail de documentation et de structuration.
- Stimuler notre autonomie, notre sérieux et notre sens des responsabilités.

Nous regrettons toutefois de ne pas avoir eu l'opportunité de visiter le site de Nanoro, ce qui nous aurait permis de mieux appréhender l'infrastructure réelle. Cependant, nous restons convaincus qu'il ne s'agit que d'une partie remise et que cette expérience pratique viendra compléter nos acquis.

II. Observations, suggestions et perspectives

1. Observations et suggestions

Nous exprimons toute notre gratitude envers l'URCN pour l'opportunité qui nous a été offerte.

Ce stage a révélé à quel point la gestion informatisée des données et processus est un enjeu majeur dans une institution de recherche comme l'URCN.

Nous suggérons a des fins d'amélioration :

- Mettre à jour la version utilisée de Dolibarr afin de bénéficier des dernières améliorations et correctifs de sécurité.
- Poursuivre le développement du module que nous avons amorcé, en l'intégrant pleinement dans le système global.
- Approfondir la documentation technique et fonctionnelle pour faciliter la prise en main par d'autres collaborateurs.

2. Perspectives

Dans une perspective d'avenir, plusieurs pistes peuvent être envisagées :

- Amélioration du module développé, avec l'ajout de nouvelles fonctionnalités adaptées aux besoins spécifiques de l'URCN.

- Optimisation de la base de données et exploration de solutions pour une meilleure gestion des performances.
- Intégration d'interfaces mobiles afin de rendre l'ERP plus accessible.
- Extension du système à d'autres processus internes de gestion (par exemple la demande de cotation).

CONCLUSION GENERALE

En définitive, ce stage de trois (03) mois à l'URCN a représenté une expérience extrêmement enrichissante. Nous avons eu l'opportunité de découvrir et de travailler sur un ERP de référence (Dolibarr), en développant un module spécifique dans un environnement professionnel exigeant. D'un point de vue technique, ce stage nous a permis d'acquérir une meilleure compréhension des architectures logicielles complexes, d'améliorer nos compétences en développement PHP et d'appréhender la gestion de bases de données volumineuses. D'un point de vue professionnel, il nous a donné un aperçu concret de la réalité du travail en entreprise, en nous obligeant à développer rigueur, autonomie et esprit d'initiative. Même si nous n'avons pas pu visiter physiquement le site de Nanoro, nous restons convaincus que cette occasion se présentera dans un avenir proche, et qu'elle viendra compléter les acquis obtenus. Enfin, ce stage nous a donné l'envie et la motivation de poursuivre notre apprentissage dans le domaine des systèmes ERP et du développement de modules personnalisés, un secteur où les besoins restent nombreux et les opportunités prometteuses.

BIBLIOGRAPHIE

- [1] Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.
- [2] Domingos, P. (2015). *The Master Algorithm*. Basic Books.
- [3] Sommerville, I. (2016). *Software Engineering* (10^e éd.). Pearson.
- [4] Pressman, R. S. (2014). *Software Engineering : A Practitioner's Approach* (8^e éd.). McGraw-Hill.
- [5] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [6] OUEDRAOGO, P. David (2021). *Mise en place d'un système de communication basé sur les SMS pour les professionnels et les particuliers*. Rapport de fin de cycle, Licence MIAGE, IBAM, Ouagadougou, 64p.
- [7] KOUDA, Ézéchias Wendtoen (2022). *Conception et réalisation d'un outil de gestion des employés de Nafann dans un contexte de gestion de projet*. Rapport de fin de cycle, Licence MIAGE, IBAM, Ouagadougou, 86p.
- [8] Kumari, S., & Pushkar, S. (2013). Comparison and Analysis of Different Software Cost Estimation Methods. *International Journal of Computer Science and Mobile Computing*, Vol. 3, Issue 6, pp. 547-557. Disponible sur : https://thesai.org/Downloads/Volume4No1/Paper_24-Comparison_and_Analysis_of_Different_Software_Cost_Estimation_Methods.pdf
- [9] Tannu, Y., & Kumar, Y. (2014). Comparative Analysis of Different Software Cost Estimation Methods. *IJCSMC*, Vol. 3, Issue 6. Disponible sur : https://www.academia.edu/7464020/Comparative_Analysis_of_Different_Software_Cost_Estimation_Methods_

WEBOGRAPHIE

- URCN – Unité de Recherche Clinique de Nanoro : <https://crun.bf>
- MSHOWTO – MVC Yapısı ve Özellikleri : <https://www.mshowto.org/mvc-yapisi-ve-ozellikleri.html>
- Forum Dolibarr ERP : <https://www.dolibarr.org/forum.php>
- Stack Overflow – Plateforme d’entraide pour développeurs : <https://stackoverflow.com>
- Documentation officielle Dolibarr : <https://wiki.dolibarr.org>
- W3Schools – Documentation PHP, MySQL, HTML, CSS : <https://www.w3schools.com>
- GitHub – Référentiels de modules Dolibarr : <https://github.com>
- PHPUnit – Outil de test unitaire utilisé pour la validation des fonctionnalités du module : <https://phpunit.de/>
- OpenAI (GPT-5.0) – Intelligence artificielle utilisée pour la recherche documentaire, l’assistance rédactionnelle et la structuration du contenu : <https://chat.openai.com/>

TABLE DES MATIERES

SOMMAIRE	i
DEDICACE	ii
REMERCIEMENTS	iii
LISTE DES SIGLES ET ABRÉVIATIONS	iv
LISTE DES FIGURES	vi
INTRODUCTION GENERALE	1
1 PRESENTATION DES STRUCTURES DE FORMATION ET D'ACCUEIL . .	2
I. Présentation de la structure de formation	2
1. Historique	2
2. Objectifs	2
3. Organisation	3
4. Filières de formation	4
II. Présentation de la structure d'accueil(URCN)	5
1. Positionnement sur le plan national	5
2. Histoire de l'URCN	6
3. Missions et objectifs de l'URCN	6
4. Organisation structurelle de l'URCN	6
2 ANALYSE ET CONCEPTION	9
I. Etude Prelable	9
1. Présentation du thème	9
2. Méthode d'analyse et conception	11
3. Groupe de Travail	16
4. Planning de réalisation	17
II. Expressions des besoins	17
1. Etude de l'existant	18
2. Description du fonctionnement attendu de l'application	20

3.	Spécifications fonctionnelles	23
4.	Spécification technique	38
III.	Conception globale	41
1.	Diagramme des classes et dictionnaire de données	41
2.	Diagramme de déploiement	48
IV.	Réalisation	49
1.	Présentation des outils de réalisation	49
2.	Présentation de l'architecture MVC de notre application	50
3.	Quelques maquettes IHM de notre application	52
4.	Tests	55
5.	Politique de sécurité	61
6.	Coût de réalisation	63
7.	Planning réel	65
3	BILAN DU STAGE	67
I.	Présentation du déroulement du stage et des activités réalisées	67
1.	Activités menées	67
2.	Connaissances acquises	67
II.	Observations, suggestions et perspectives	68
1.	Observations et suggestions	68
2.	Perspectives	68
	CONCLUSION GENERALE	70
	BIBLIOGRAPHIE	71
	WEBOGRAPHIE	72
	TABLE DES MATIERES	73
	ANNEXES	I
	Annexe 1 : UML – Langage de modélisation unifié	I
	Annexe 2 : Méthodes d'estimation du coût logiciel	II
	Annexe 3 : Apache – Serveur HTTP	VI
	Annexe 4 : MySQL – Système de gestion de base de données	VI
	Annexe 5 : XAMPP – Environnement de développement local	VI
	Annexe 6 : OpenSign – Signature électronique	VI
	Annexe 7 : Formulaire d'expression de besoin	VII

ANNEXES

Annexe 1: UML – Langage de modélisation unifié

UML (Unified Modeling Language) est un langage de modélisation graphique standardisé, utilisé pour représenter la conception et l'architecture d'un système logiciel. Il permet de spécifier, visualiser, documenter et construire les composants d'un logiciel, en particulier dans le cadre du développement orienté objet.

Depuis sa version 2.0, UML propose 13 types de diagrammes, répartis selon les phases du cycle de développement :

- **Diagramme des cas d'utilisation** : modélisent les besoins fonctionnels exprimés par les utilisateurs.
- **Diagramme de séquence (vue externe)** : illustre les interactions entre les acteurs et le système.
- **Diagramme d'activité** : décrit les enchaînements d'actions ou de processus.
- **Diagramme des classes** : représente la structure interne du logiciel (attributs, méthodes, relations).
- **Diagramme d'objet** : montrent l'état des objets à un instant donné.
- **Diagramme d'état-transition** : modélisent les changements d'état d'un objet en réponse à des événements.
- **Diagramme de séquence (vue interne)** : détaillent les interactions internes entre composants.
- **Diagramme de composant** : présente les modules physiques du logiciel.
- **Diagramme de déploiement** : illustre la distribution du logiciel sur l'infrastructure matérielle.

Le choix des diagrammes à utiliser dépend du type de projet, de sa complexité et des objectifs de modélisation. Dans le cadre de notre projet, les diagrammes des cas d'utilisation, des classes et de séquence ont été privilégiés pour représenter les interactions et la structure du module développé.

Annexe 2: Méthodes d'estimation du coût logiciel

Cette annexe présente en détail les deux méthodes d'estimation utilisées pour évaluer le coût de développement du module de gestion des expressions de besoins.

Méthode COCOMO – Estimation algorithmique

La méthode COCOMO (Constructive Cost Model) est une approche algorithmique permettant d'estimer le coût, l'effort et la durée de développement d'un logiciel. Elle repose sur le volume de code à produire (exprimé en Kilo Lignes Sources – KLS) et la complexité du projet.

COCOMO distingue trois types d'applications :

- Type S (Simple) : applications déterministes, peu de cas particuliers, contraintes limitées.
- Type P (Intermédiaire) : complexité modérée, davantage de tests et de cas particuliers.
- Type E (Complexe) : systèmes temps réel, interfaces riches, nombreuses contraintes et incertitudes.

Les formules de calcul sont les suivantes :

Complexité	Effort (en homme-mois)	Temps de développement (TDEV en mois)
S	$\text{Effort} = 2,4 \times (\text{KLS})^{1,05}$	$\text{TDEV} = 2,5 \times (\text{Effort})^{0,38}$
P	$\text{Effort} = 3 \times (\text{KLS})^{1,12}$	$\text{TDEV} = 2,5 \times (\text{Effort})^{0,35}$
E	$\text{Effort} = 3,6 \times (\text{KLS})^{1,20}$	$\text{TDEV} = 2,5 \times (\text{Effort})^{0,32}$

TABLE 3.1 – Formules de calcul selon la méthode COCOMO

Application au projet

Pour notre module de gestion des expressions de besoins :

- Type d'application : S (Simple) - module métier standard
- Volume de code estimé : 6 000 lignes → KLS = 6
- Salaire mensuel moyen : 250 000 FCFA

Calculs détaillés

- Effort = $2,4 \times 6^{1,05} = 2,4 \times 6,62 = 15,88$ hommes-mois
- Durée (TDEV) = $2,5 \times 15,88^{0,38} = 2,5 \times 2,65 = 6,63$ mois
- Effectif moyen = $15,88 \div 6,63 = 2,39$ développeurs
- Coût développement = $15,88 \times 250\,000 = 3\,970\,000$ FCFA

Méthode par analogie – Estimation basée sur la réalisation effective

La méthode par analogie s'appuie sur des références de projets similaires. Dans notre cas, nous disposons de la meilleure référence possible : le développement effectif du module durant la période de stage de 3 mois avec un état d'avancement de 90%.

Réalisation effective du projet

- Période de développement : 3 mois de stage
- État d'avancement : 90% des fonctionnalités développées
- Compétences initiales : Bon niveau PHP, première expérience Dolibarr
- Réalisation effective : Toutes les fonctionnalités clés opérationnelles

Calcul basé sur la productivité réelle

Élément	Valeur	Calcul
Temps réel de développement	3 mois	Période de stage
Avancement atteint	90%	Fonctionnalités développées
Temps total extrapolé	3,33 mois	$3 \text{ mois} \div 90\%$
Marge pour tests/réglages	+15%	Bonnes pratiques
Durée finale estimée	3,8 mois	$3,33 \times 1,15$

TABLE 3.2 – Calcul basé sur la réalisation effective

Validation par les fonctionnalités développées

Durant les 3 mois de stage, les fonctionnalités suivantes ont été entièrement développées :

- CRUD complet des expressions de besoin
- Workflow de validation multi-niveaux
- Gestion des utilisateurs et droits d'accès
- Génération de documents PDF automatisée
- Tableaux de bord statistiques
- Intégration Dolibarr complète

Comparaison avec les références du Dolistore

Le Dolistore est la boutique officielle de modules pour l'ERP Dolibarr. Il regroupe des extensions développées par la communauté ou des prestataires, permettant d'ajouter des fonctionnalités spécifiques à Dolibarr. Ces modules sont accessibles en ligne, accompagnés de descriptions techniques, de captures d'écran et parfois d'indications sur leur niveau de maturité ou leur durée de développement.

Dans le cadre de ce projet, plusieurs modules disponibles sur le Dolistore ont été consultés à titre de référence, notamment ceux liés à la gestion des demandes internes, des achats ou des processus de validation. À partir de ces observations, il est possible de situer le projet dans une fourchette indicative :

- **Durée de développement observée** : les modules similaires affichent des temps de réalisation compris entre 3 et 5 mois.
- **Productivité constatée** : malgré une première prise en main de Dolibarr, le rythme de développement est resté soutenu.
- **Compétences mobilisées** : notre niveau de développement en langage PHP nous a aidé à nous adapter à l'environnement Dolibarr.
- **Niveau de complexité** : les fonctionnalités développées présentent une complexité comparable à celle de certains modules du Dolistore.

Analyse de l'écart avec l'estimation COCOMO

L'estimation obtenue via le modèle COCOMO (6,63 mois) diffère sensiblement de la durée réelle de développement observée (3,8 mois). Plusieurs facteurs peuvent expliquer cet écart :

- **Nature du modèle COCOMO** : ce modèle est conçu pour des contextes industriels, avec des équipes pluridisciplinaires, des processus formalisés et des charges organisationnelles importantes.
- **Contexte du projet** : le développement a été réalisé par une seule personne, ce qui a permis une exécution plus directe et moins fragmentée.

- **Allègement des charges organisationnelles** : l'absence de réunions d'équipe, de reporting complexe ou de coordination interservices a réduit les délais non techniques.
- **Focalisation du travail** : le projet a bénéficié d'une concentration continue sur un seul objectif, sans interruptions liées à d'autres tâches ou projets.
- **Réutilisation de composants existants** : l'intégration dans Dolibarr a permis de capitaliser sur des éléments déjà disponibles, réduisant ainsi le volume de développement nécessaire.

Cette analyse ne remet pas en cause la pertinence du modèle COCOMO, mais souligne que son application dans certains contextes peut conduire à des écarts significatifs. Elle permet également de mieux comprendre les leviers d'optimisation mobilisés dans le cadre de ce projet.

Synthèse comparative

Aspect	COCOMO	Réalisation effective
Durée estimation	6,63 mois	3,8 mois
Coût développement	3 970 000 FCFA	950 000 FCFA
Base de calcul	Algorithmique	Réalisation mesurée
Adéquation contexte	Faible	Excellente
Preuve concrète	Non	Oui (90% développé)

TABLE 3.3 – Comparaison des approches d'estimation

Conclusion méthodologique

L'estimation basée sur la réalisation effective est retenue car :

- Elle s'appuie sur des données réelles de développement
- Elle démontre une productivité exceptionnelle malgré la nouveauté de Dolibarr
- Elle est vérifiable par l'état d'avancement du module
- Elle reflète la valeur réelle du travail accompli
- Elle confirme l'efficacité des compétences PHP pour compenser l'apprentissage

L'estimation de 3,8 mois et 950 000 FCFA représente donc la valorisation juste et défendable du développement effectif, basée sur notre performance réelle durant le stage.

Annexe 3: Apache – Serveur HTTP

Apache est l'un des serveurs web les plus utilisés dans le monde. Il permet de gérer les requêtes HTTP et de servir des contenus dynamiques ou statiques aux utilisateurs. Dans le cadre du développement d'applications web, Apache joue le rôle d'intermédiaire entre le navigateur et les scripts exécutés côté serveur (comme PHP).

Il est souvent intégré dans des environnements de développement comme XAMPP, facilitant ainsi la mise en place d'un serveur local pour tester les applications avant leur déploiement en production.

Annexe 4: MySQL – Système de gestion de base de données

MySQL est un système de gestion de base de données relationnelle (SGBDR) open source. Il permet de stocker, organiser et interroger des données de manière structurée à l'aide du langage SQL.

Dans les projets ERP comme Dolibarr, MySQL est utilisé pour gérer les informations liées aux utilisateurs, aux modules, aux expressions de besoins, aux droits d'accès, etc. Sa robustesse et sa compatibilité avec PHP en font un choix privilégié pour les applications web professionnelles.

Annexe 5: XAMPP – Environnement de développement local

XAMPP est un package logiciel regroupant plusieurs outils essentiels pour le développement web : Apache (serveur HTTP), MySQL (base de données), PHP (langage serveur) et Perl. Il permet de simuler un environnement serveur complet sur une machine locale.

Grâce à XAMPP, les développeurs peuvent tester leurs applications sans avoir besoin d'un serveur distant. Il est particulièrement utile pour les projets comme Dolibarr, qui nécessitent une configuration serveur pour fonctionner correctement.

Annexe 6: OpenSign – Signature électronique

OpenSign est une bibliothèque ou un module utilisé pour intégrer la signature électronique dans les systèmes d'information. Elle permet d'authentifier les actions réalisées par les utilisateurs, notamment dans les processus de validation ou de traitement.

Dans le contexte d'un ERP comme Dolibarr, OpenSign peut être utilisé pour garantir la traçabilité des opérations sensibles, comme la validation d'une expression de besoin ou la signature d'un document PDF. Elle renforce la sécurité et l'intégrité des données en associant chaque action à une identité numérique vérifiable.

Annexe 7: Formulaire d'expression de besoin

Cette annexe présente une capture du formulaire PDF actuellement utilisé par l'URCN pour la soumission des expressions de besoin. Ce document constitue le point de départ du processus de validation et d'approvisionnement.

Structure du formulaire

Le formulaire comporte les sections suivantes :

- Informations sur le demandeur : nom, prénom, service ou projet concerné
- Détails de l'expression : tableau des items demandés avec désignation, référence, quantité
- Liste de trois fournisseurs recommandés
- Zone de validation : signatures électroniques des responsables
- Zone d'approvisionnement : champs complétés par le service achat

Illustration

 DRCO	FORMULAIRE	Reference : FO_07_001/01 Version : 00 Date application : 15/02/2023
Unité de Recherche Clinique de Nanoro		
EXPRESSION DE BESOIN		

DEMANDEUR (Nom et Prénom): 	Date : 	Page :
SERVICE / PROJET :	IMPUTATION : (à renseigner si connu)	

Item n°	DESIGNATION	REFERENCE*	QUANTITE

Fournisseurs recommandés	
F1	
F2	
F3	

* joindre les spécifications techniques si nécessaire

Visa responsable de service/projet demandeur (pour approbation du besoin)	Visa du gestionnaire de crédit (pour confirmation de la disponibilité financière)
Nom & Prénom :	Nom & Prénom :
Date :	Date :
Signature :	Signature :

Cadre réservé au service Achat/Appro

Numéro EB : aaaa mm num	Expression reçu le : dd mmm aaaa
Traité par :	DC transmises le : dd mmm aaaa
Signature :	Numéros des DC transmises:

Urgence : Normale

** Demande de Cotation

FIGURE 3.1 – Formulaire d’expression de besoin utilisé à l’URCN

Ce formulaire est éditable et circulant par voie électronique. Il est signé via OpenSign, puis enrichi par le service achat dans Dolibarr. Son format actuel présente des limites en matière de sécurité et de traçabilité, comme détaillé dans la section précédente.